US Tax

A formally verified income tax calculator

Arnab Ghosh · Hackathon submission, November 2026 · github.com/arniber21/ustax

What "verified" means here

Judges' summary

- **The law is the code.** The federal 1040 rules for tax year 2024 are written in Catala: 37 modules, 20,802 lines, with the statute and IRS instructions quoted next to each rule. The Catala compiler type-checks all of it, and 602 named tests pass.
- **It found 33 places where the IRS's own reference engine (Direct File) does not follow the law.** One (DF-BUG-8) is confirmed by running the IRS engine; the others are cited candidates. 25 were found while writing the statute as Catala next to the Direct File XML, 7 in an earlier review of the Direct File code, and 1 from running Direct File's own test returns. **The type checker itself found none of them**; what it did is make the rules precise enough to compare line by line.
- **Lean then proves the arithmetic.** The calculator that runs on this site is written in Lean, with 9 checked theorems about the whole return and 31 more that tie its pieces to the statute. Lean and Catala are linked by differential testing, not by a proof.

The headline

The unusual part of this project is the Catala layer, not the Lean layer. The tax law is written as executable code that reads like the statute: each section of the Internal Revenue Code or the IRS instructions is quoted, and the rule that implements it sits directly under the quote. The Catala compiler checks the types and the structure of every rule. The rules then run against the IRS's own engine on its own test returns.

Writing the law down this precisely, and checking it against the IRS engine, showed where the IRS reference implementation disagrees with the statute or the instructions. The log has 33 entries (Direct File bugs). The calculator follows the law in 30 of them; for 3 (DF-BUG-2, 4 and 10) it keeps Direct File's behaviour until the project owner decides. Every entry is pinned by a test.

The Lean proofs come after that. They do not say the law is right. They say that the calculator's arithmetic has properties that hold for every input: for example, you never get a refund and a bill on the same return.

	Verified in Catala (the headline)	**Proved in Lean**
What it is	The tax rules written as readable, executable law, checked by the Catala compiler, named tests and the IRS engine	A theorem about the calculator's own code, checked by the Lean kernel
Strength	**Evidence**: it agrees with the IRS reference engine on every scenario we have, except where we follow the statute on purpose (each case logged)	A **proof**: it holds for every input, not only the ones we tried

	Verified in Catala (the headline)	**Proved in Lean**
Example	"Our rules give the same answer as IRS Direct File on 157 sample returns"	"You never get both a refund and a bill"
What it found	33 places where Direct File disagrees with the law	Nothing about the law. It locks the arithmetic of the calculator.
Can it be wrong?	Yes: a rule can be wrong in a way that no sample return exercises	Only if the theorem statement itself says the wrong thing (a human reviews the statements)

Proved = a Lean theorem holds for every input. **Verified** = the Catala rules agree with IRS Direct File on every scenario we have. It is not a proof.

What the Catala compiler and type checker catch

Measured on Catala 1.3.0. Each error message below comes from running `catala typecheck` or `catala interpret` on a small file written for this page.

Check	**When**	**What happens**	**Message (Catala 1.3.0)**
Static types. `money` (whole cents), `decimal` (exact fractions), `integer`, `date`, `boolean`, records and enumerations are different types.	Compile time	Rejected	`I don't know how to apply operator + on types money and decimal`
Exhaustive matches. A `match` on an enumeration must handle every case, for example all five filing statuses.	Compile time	Rejected	`The constructor HeadOfHousehold of enum Status is missing from this pattern matching`
Dependency cycles. No value can depend on itself, even through a sub-scope (a module call).	Compile time	Rejected	`Cyclic dependency detected between the following variables of scope Return: agi → interest → agi`
Undefined or unused variables.	Compile time	Warning only	`the variable "amount" is declared but never defined`; `Unused variable: age does not contribute to computing any of scope S outputs`
Conflicting rules. Two definitions of one variable apply at the same time, and neither is declared an `exception` to the other.	**Run time**, only on an input that triggers both	Evaluation stops	`conflict between multiple valid consequences for assigning the same variable`
No applicable rule. No definition applies to an input.	**Run time**	Evaluation stops	`no applicable rule to define this variable in this situation`

Catala 1.3.0 does **not** find overlapping or missing rules at compile time. A `Conflict` or a `NoValue` shows up only when a test or a difftest input reaches it. This page used to say that the compiler "flags overlapping or conflicting exceptions"; that was too strong.

How this code uses the default logic. The modules do not use Catala's `exception` and `label` mechanism (0 occurrences). Each scope variable has exactly one definition: 1,417 definitions in the 37 modules, none of them a second definition of the same variable (counted by a script on 2026-11-01). Choices are written as `if`/`match` inside that one definition. So a `Conflict` cannot happen in this code. A `NoValue` can, from a conditional definition with no default, and none happened in the 602 tests or in the difftest runs.

What the compiler caught or forced in this repository

The history does not record individual type errors. Agents fixed those before committing, and the PRs report only "typecheck OK". Today `clerk typecheck` on all 37 modules and 42 test files gives "Typechecking successful!" with one deprecation warning (2026-11-01). The cases below are the ones the PRs and the source comments record.

What was written	What the compiler said or forced	How it was resolved	Where
Form 1040 passed the real AGI into the single `Interest` module call. `Interest` needs AGI only for three knockouts, but its interest amount feeds AGI.	A dependency cycle. Catala treats a module call as one node, so one call is a cycle. The PR says "a dependency cycle (verified)".	Two calls: `interest_ko` gets the real AGI and gives only the knockouts; `interest` gets `$0` and gives the amounts.	`form_1040.catala_en` "Wiring the sub-scopes", PR #94
Whole return: the HSA knockout reads Social Security line 6a, and line 6b reads the HSA deduction.	A cycle between the two modules.	A second call of the Social Security module, used only for the knockout.	PR #133, `calculator.catala_en`
Whole return: Form 2441 Part III gives line 1e (taxable care benefits, part of income), but the credit needs AGI and tax.	A cycle through income.	Part III is called first with AGI and tax at `$0`, and the credit call gets the real values. A run-time `assertion` checks that both calls give the same line 1e.	PR #133, `calculator.catala_en` (the one `assertion` in the modules)
Filing status depends on head-of-household and surviving-spouse qualifying persons, which depend on the dependent tests, which depend on whether the filers file jointly.	A cycle, unless one link is cut. Catala has no fixed point, so a staged design was needed.	The filing status is an input. A status the household does not qualify for is a knockout.	`CONVENTIONS.md` §7, PR #85, PR #128
The 1099-R distribution code as a list of `CodeChar` enumeration values.	Each code letter has to be a named case, and every `match` handles them all.	Codes are read as a whole. Unsupported combinations (`7D`, `1`, ...) are knockouts (DF-BUG-1).	`form1099_rs.catala_en`, PR #102
Form 8962 line 5 (household income as a percent of the poverty line) as an integer.	Catala 1.3 has no cast from `decimal` to `integer`.	The value is kept as a `decimal` with a whole value.	`ptc_family.catala_en`
A `but replace` record update nested inside a module-call record.	Not a type error: a **bug in Catala 1.3's OCaml backend**. `clerk run` generated OCaml that does not compile.	The updates were moved into their own internal variables (`primary_disability`, `secondary_disability`).	PR #133

Forced explicit decisions

The type system and the literate style also forced decisions that a spreadsheet or a loosely typed engine can leave implicit:

- **Units and rounding.** `money` is whole cents and `decimal` is an exact fraction, so every place a dollar amount is multiplied by a rate has to say how it rounds. Each Direct File `Round` became one explicit `R.round_dollar` call with the DF fact path in a comment, and the rounding helpers have their own tests (`CONVENTIONS.md` §4). This is how three rounding decisions were made and recorded: per-item rounding (DF-BUG-4), cent ties (Q-R2) and double rounding (Q-R3).
- **Exact arithmetic.** `decimal` is exact, so Direct File's conversion of a fraction to a dollar amount through a 32-bit `Float` (DF-BUG-5) could not be copied by accident. Matching it would have needed extra code on purpose.
- **Every deviation is marked.** A rule that keeps a Direct File choice the law does not make has a `# QUIRK(Qn)` comment. A rule that follows the law against Direct File has a `# DF-BUG(n)` comment and a citation (94 such markers in the modules). Each one is listed in Quirk decisions with the test that pins it.
- **Inputs are complete, and functions are total.** A situation the rules cannot compute is an explicit knockout with a reason. The rules never guess.

Disagreements with the law that formalising in Catala surfaced

We logged 33 places where Direct File (the IRS's open-source reference implementation, "DF" below) disagrees with the statute or the IRS instructions. The calculator follows the law in 30 of them (including knocking out a return it cannot compute); for DF-BUG-2, 4 and 10 it keeps Direct File's behaviour until the project owner decides.

Discovered by Arnab Ghosh's ustax project. Full log with the XML line numbers: Direct File bugs. The decision for each entry and the test that pins it: Quirk decisions.

How they were found (counted from the table below; source: the PR bodies and the `# DF-BUG(n)` comments in `catala/*.catala_en`):

How found	Count	Entries
Encoding: writing the rule in Catala with the statute quoted, and the DF XML open side by side	25	7, 9, 10, 12 to 33
Survey: flagged in the review of the DF code before any Catala was written (`scope.md` §11 and §3.1), then checked against the XML and the law while writing the Catala conventions (PR #85)	7	1 to 6, 11
Oracle run, confirmed in the engine: DF knocked out 4 of its own test returns; tracing it led to the bug, which was confirmed by evaluating the fact in DF's engine	1	8
Catala compiler or type checker	**0**	none

For 8 entries (3, 8, 13, 14, 15, 19, 23, 24) the differential test also shows the difference on real DF test returns, listed as `knownDivergence` entries in `difftest/mappings/*.mjs`. The other 25 do not change any value in DF's test returns and are pinned by unit tests only.

> **Status.** Exactly one entry is **confirmed by running the IRS engine**: DF-BUG-8. DF-BUG-5 is an arithmetic artefact read in the engine's source (`types/Dollar.scala`), not a tax rule; the log marks it "confirmed (engine)". The other 31 are **candidates**: each has a citation and a test, but no one outside the project has confirmed them. (The log marks DF-BUG-1 and DF-BUG-7 as "fixed", which means our code follows the law; the DF behaviour is still a candidate.)

ID	What Direct File does	What the law says	Status	How found	Catala
DF-BUG-1	**1099-R codes.** Strips characters from the distribution code, so `7D` is accepted as `7`,	The IRS 1099-R box 7 table: each code has its own meaning; `7D` is not `7`	candidate (fixed)	Survey (#85); encoded in #102	form1099_rs:181

ID	What Direct File does	What the law says	Status	How found	Catala
	and `1` is knocked out as empty				
DF-BUG-2	**Education credit.** American opportunity credit is a `$0` stub with no knockout	IRC 25A: eligible filers get the credit	candidate (DF kept, owner to decide)	Survey (#85)	form_1040:511
DF-BUG-3	**Qualifying child.** No tie-breaker when two taxpayers can claim a child	IRC 152(c)(4)	candidate	Survey (#85); difftest: 1 return	qualifying_child:574
DF-BUG-4	**Rounding.** Rounds each item, then the sum	1040 instructions p 23: round only the total	candidate (DF kept, owner to decide)	Survey (scope.md §3.1)	form_w2s:250
DF-BUG-5	**Engine arithmetic.** Converts a fraction to dollars through a 32-bit `Float`	Exact arithmetic (not a tax rule)	engine source	Survey (scope.md §11)	test_rounding:78
DF-BUG-6	**Total tax.** Schedule 2 line 21 is defined but not added to total tax	Form 1040 line 24 = line 22 + line 23	candidate (same value today)	Survey (#85)	form_1040:447
DF-BUG-7	**Student loan interest.** Rounds the phase-out ratio to 3 decimals, then rounds twice more	IRC 221(b)(2)(B): exact ratio	candidate (fixed)	Encoding, #101	student_loan_adjustment:82
DF-BUG-8	**1099-R box 8.** `StripChars(text, "0.")`: its XML reader turns the literal `0.` into the number `0`, so `0.00` looks nonzero. **Wrongly knocks out 4 of Direct File's own test returns.**	A zero percentage is no knockout	**confirmed, engine run**	Oracle run; traced in #102, engine-checked in #109	form1099_rs:485
DF-BUG-9	**1099-MISC.** Treats every 1099-MISC as Alaska Permanent Fund dividend income	Schedule 1 line 8g is only for the Alaska PFD	candidate	Encoding, #100	form1099_misc:75
DF-BUG-10	**Unemployment.** Caps the in-year repayment at box 1 on each form	The 1040 instructions have no such cap	candidate (DF kept, owner to decide)	Encoding, #100	unemployment:96
DF-BUG-11	**Qualifying child.** A disabled child born on the same day as the filer (a twin) fails the age test	IRC 152(c)(3)(B): a disabled person passes at any age	candidate	Survey; pinned while re-reading the XML (#85); encoded in #103	qualifying_child:406
DF-BUG-12	**HSA.** Splits the family limit by what each spouse contributed	IRC 223(b)(5)(B)(ii): split equally unless agreed	candidate	Encoding, #104	hsa:1117

ID	What Direct File does	What the law says	Status	How found	Catala
	($7,000 where the law gives $5,150)				
DF-BUG-13	**Schedule R.** Adds cost recovery and rollovers to line 13b	IRC 22(c)(3)(A), Schedule R instructions p R-5	candidate	Encoding, #106; difftest: 33 checks in 21 returns	elderly_and_disabled:30
DF-BUG-14	**Saver's credit.** Zeroes an ineligible spouse's distributions in both columns	IRC 25B(d)(2)(D), Form 8880 line 4	candidate	Encoding, #107; difftest: 2 returns	savers_credit:371
DF-BUG-15	**Qualifying relative.** Counts the filer's own qualifying child as a qualifying relative	IRC 152(d)(1)(D): "not a qualifying child of such taxpayer"	candidate	Encoding, #108; difftest: 2 persons	qualifying_relative:49
DF-BUG-16	**Qualifying relative.** Reads "qualifying child of another taxpayer?" only behind its own age gate	IRC 152(c), (d)(1)(D)	candidate	Encoding, #108	qualifying_relative:130
DF-BUG-17	**Earned income credit.** Combat-pay variants pay $1 to $2 where the credit is $0	IRC 32: $0 with no earned income or past the phase-out	candidate	Encoding, #112	eitc:306
DF-BUG-18	**Divorced parents.** Never asks about a multiple support agreement	IRC 152(e)(5): no Form 8332 split under one	candidate	Encoding, #115	dependents_benefit_split:50
DF-BUG-19	**Head of household.** "Considered unmarried" through any child, claimable or not	IRC 7703(b)(1): only a child the filer can claim	candidate	Encoding, #121; difftest: 1 person, 4 filing-status graphs (#128)	hoh_qss_persons:66
DF-BUG-20	**Head of household.** The dependent must actually be claimed	IRC 2(b)(1)(A)(ii): the filer must be entitled to claim	candidate	Encoding, #121	hoh_qss_persons:519
DF-BUG-21	**Head of household.** Excludes a married child also for no TIN or a custody release	IRC 2(b)(1)(A)(i): only joint return or citizenship	candidate	Encoding, #121	hoh_qss_persons:135
DF-BUG-22	**Head of household.** Admits a relative the filer cannot claim (custody split)	IRC 2(b)(1)(A)(ii)	candidate	Encoding, #121	hoh_qss_persons:378
DF-BUG-23	**Child care credit.** A foster parent or grandparent gets the credit through the Form 8332 split (credit $702 where the law gives $602)	IRC 21(e)(5), Form 2441 instructions p 1, Pub 503 p 4: custodial parent only	candidate	Encoding, #129; difftest: 6 facts, 1 return	cdcc:41

ID	What Direct File does	What the law says	Status	How found	Catala
DF-BUG-24	**Child care credit.** AGI of $0 or less gives a 0 percentage	IRC 21(a)(2): 35%	candidate	Encoding, #129; difftest: 3 facts, no credit changes	cdcc:45
DF-BUG-25	**Child care credit.** Married filing separately exception tests two possibly different people	IRC 21(e)(4): one household	candidate	Encoding, #129	cdcc:47
DF-BUG-26	**Child care credit.** Surviving spouse's line 19 comes out `$0`	Form 2441 line 19: "all others" use line 18	candidate	Encoding, #129	cdcc:51
DF-BUG-27	**Child care credit.** Ignores forfeited benefits unless W-2 box 10 is positive	Form 2441 line 14 covers line 12 or 13	candidate	Encoding, #129	cdcc:54
DF-BUG-28	**Child care credit.** Requires the filer to have chosen to claim the child	IRC 21(b)(1)(A): a dependent the filer may claim	candidate	Encoding, #129	cdcc:56
DF-BUG-29	**Earned income credit.** The qualifying child must be claimed	IRC 32(c)(3): no claim required	candidate	Encoding, #133	calculator:389
DF-BUG-30	**Earned income credit.** A married child must be a claimed dependent	IRC 32(c)(3)(B): the filer need only be entitled to claim	candidate	Encoding, #133	calculator:393
DF-BUG-31	**Premium tax credit.** A policy starting December 1 never counts December ($0 where the law gives $500)	Form 8962 instructions: in effect on the 1st counts	candidate	Encoding, #135	ptc_allowed:543
DF-BUG-32	**Premium tax credit.** Uses annual totals for a January-to-December policy with fewer than 12 coverage months ($6,000 where the law gives $5,500)	Form 8962 instructions: annual only with all 12 months	candidate	Encoding, #135	ptc_allowed:558
DF-BUG-33	**Premium tax credit.** Annual totals if any Form 1095-A qualifies	Form 8962 instructions: each plan	candidate	Encoding, #135	ptc_allowed:770

Line numbers in the Catala column are on `main` on 2026-11-01. To find a marker after the files change, search for `DF-BUG(n)`. Some older comments use provisional labels; see Questions 8 in Quirk decisions.

How Catala is checked against the IRS

1. **The type checker.** `clerk typecheck` on all 37 modules and 42 test files: "Typechecking successful!" (2026-11-01, Catala 1.3.0; one deprecation warning). It runs in `make check` and in CI.

2. **Named tests.** `clerk test`: 602 of 602 test scopes pass (2026-11-01). Each module has hand-worked examples from the forms and instructions, tests that a known-wrong output is rejected, and one test per DF-BUG entry that asserts the law's value and not Direct File's (Quirk decisions).
3. **Differential test against the Direct File engine.** The IRS Direct File project ships 161 test returns. We run its engine on them to get the expected value of every fact (`oracle/facts.csv`), then run the whole-return Catala scope on the same answers and compare. 4 returns are knocked out by DF itself (all 4 are DF-BUG-8), so 157 are compared. Result for `make difftest SCOPE=calculator`: **157 of 157 returns and 5,360 of 5,360 fact checks pass** (754 oracle values are incomplete and not checked). 9 checks are reported apart as known divergences and never counted as passes: DF-BUG-3, DF-BUG-23, and one return whose chosen filing status DF's own validity check rejects. Source: PR #136, re-run on 2026-11-01.

The runner, its mappings and the knocked-out scenarios are described in `catala/DIFFTEST.md`.

What Lean adds

The calculator that runs behind this site is written directly in Lean. Statements about it are checked by Lean's kernel, with no `sorry` and no extra axioms.

- **Comparator-checked statements** (`lean/comparator.json`, 9 theorems): the tax on taxable income never decreases as taxable income increases with filing status fixed, and is never negative; line 16 equals the printed IRS Tax Table for every cent below $100,000 and the Tax Computation Worksheet from $100,000 through $1 trillion; refund and amount owed are both non-negative, at most one is non-zero, and their difference is payments minus tax; credits never exceed the tax they offset; the earned income credit stays between zero and its maximum.
- **Additional statements** (`lean/comparator.extra.json`, 31 theorems): for example, the standard deduction, interest, W-2 totals, Social Security and the adjustments equal a plain statement of the statute for every input, and on the whole return `supported = false` exactly when the list of reasons is not empty.
- **The checks around them.** `make audit` rejects `sorry`, new axioms and `native_decide`, and scans for vacuous or contradictory statements. `make comparator` replays the proofs in a sandbox against a fixed file of statements.

> **The honest limit.** The Lean calculator and the Catala rules are linked by **differential testing, not by a proof**: both run on the same inputs and the results are compared. A Lean theorem proves a property of the Lean code. It does not prove that the Lean code computes the Catala rules, or that the Catala rules are the law. Catala to Lean explains why we did not build a translator, and what that choice costs.

See Spec surface for the exact definitions each theorem depends on, and Comparator and lake check for how the checks run.

What you still have to trust

- The Lean kernel, and the Lean server and JSON reader that sit between your browser and the calculator.
- The theorem **statements** (the spec files). A proof only shows the code meets the statement, so a human has to confirm the statements say what they should. See Spec surface.
- The Catala rules as a reading of the law. They are written next to the quoted text so that a person can check them, but no tool checks that a rule means what the statute means.
- The IRS Direct File engine, as the reference we compare against. It is a government project, but it has bugs, which is why we log them.
- The Catala toolchain, but only for tests: the running calculator does not use it.

What is not covered

Federal income tax for tax year 2024 only, with the standard deduction only. Anything outside the supported situations is refused with a reason, not guessed. Nothing is filed, and this is not tax advice.

Income Tax Calculator

Overview

`ustax` is a formally verified income tax calculator. It leverages a formalization of a subset of US income tax law in the Catala DSL to ensure that tax calculation business logic is consistent with the law as written. `ustax` supports the subset of US tax law implemented in the IRS' Direct File, whose source is publicly available on GitHub. Formalized tax calculations in Catala are converted (via an LLM, but ideally, through transpilation) into Lean code, where we prove additional properties (for example, that tentative tax never decreases as taxable income rises).

Catala

Catala is a DSL for the law. It allows people to write laws with code. Crucially, it removes ambiguity in interpretation and enables type-checking. For the subset of the law that deals with numerics, calculations, logic, etc., Catala can be transpiled into either OCaml or Java for automated decision-making and calculation. The French government used Catala to translate, verify, and derive automated calculations for both income tax and housing benefits.

Thus, the entry point for developing `ustax` was first to create the requisite Catala formalization. To do so, I pointed a swarm of agents at a source dump of Direct File. They were first to establish a set of tax laws that they could infer from the program; separately, subagents would search through US revenue code and find the actual tax law based on the inferred specification. Next, another wave of subagents would collaboratively formalize the tax laws in Catala. With a comprehensive Catala specification in hand, I then turned to Lean.

Lean

Lean is the language of choice for this hackathon. Furthermore, it would enable supplemental verification to Catala's guarantees. Unfortunately, Catala only supports code generation to OCaml, Java, JavaScript, Python, and C. Ideally, we would write a backend for Lean – unfortunately, an entire compiler backend to a proof verifier is quite a bit to chew off for a weekend hackathon, so I decided to use agents to guide the translation from Catala to Lean. However, in an ideal world, this would be automated.

The generated Lean business logic was augmented with a suite of additional correctness theorems for tax calculations, including matching figures between calculation areas, credit calculations, EITC calculations, and more. Finally, I integrated the module with a typical Lean web-server setup with Lean's HTTP library (opportunity: a web server framework for Lean, when we don't want to deal with complexity). Finally, we paired it with a basic frontend written in React and Typescript.

Trusted Base

Unfortunately, this program has quite a few trusted bases: the Lean compiler, libraries, JSON parsing, etc. A more fleshed-out non-hackathon version could certainly tackle many of these. In particular, we rely on a correct Catala-to-Lean translation – this should be automated with a compiler. We also rely on correct interpretation of the law text to Catala – while this is generally subjective, US tax law is usually explicit (though less so than other countries). A fuller version would remove these from the trusted base by compiling Catala to Lean and by having people review the formal spec against the statute.

US Tax: the tax law as checked code, and a calculator with proofs

Hackathon write-up. Numbers are as of 2026-11-01, and each says where it came from. The short, non-technical version is What "verified" means here; the author's own account is How we built this.

Why

Tax software is a program that tells you how much money you owe the government. People trust it because they cannot check it. Mistakes happen in commercial and government software alike, and when they do, they repeat on every return that takes the same path.

The tax code is also unusually well suited to being written as code. It is long, but each rule is small and precise: "line 15 is line 11 minus line 14, but not less than zero." The rules as code movement argues that the law should be published in a form a computer can run, so that everyone runs the same rules. Catala, a programming language built for writing laws, came out of that movement.

We wanted to do two things with that idea. First, write the 2024 federal income tax rules as code that a tax expert can read next to the statute, and check them against the IRS's own implementation. Second, run a calculator whose arithmetic is **proved**, not just tested, and say exactly where the line between the two falls.

What we built

US Tax is a web app that computes a 2024 US federal income tax return (Form 1040) for the kinds of returns IRS Direct File handled: wages, interest, unemployment, Social Security, retirement income, the common adjustments, the standard deduction, the child tax credit, the earned income credit, the child and dependent care credit, the saver's credit, the credit for the elderly or disabled, and the premium tax credit. You answer questions in plain language; it shows your refund or balance due, line by line.

Each line has a **Law & proof** drawer: the statute and IRS instructions it comes from, the Catala rule that computes it, the Lean theorems about it, and the tests that pin it (`api/provenance.json`, 19 line items, checked current by `make provenance-check`). If your return is outside its scope, it stops and says why, instead of guessing. Direct File calls this a knockout.

The app also has a docs section at `/docs` (markdown files in `docs/site/` and a few others), an interactive API reference at `/docs/api`, a ⌘K command palette, and a Verification page whose numbers a unit test checks against the repository (PR #155).

Under the web app there are three layers:

```
Law (forms, instructions, IRC, Rev. Proc. 2023-34; copies in sources/)
  -> Catala rules        37 modules, the statute quoted next to each rule; the test oracle
  -> Lean code + proofs  the calculator that runs, and theorems about it
  -> Lean HTTP server    Std.Http from Lean core, plain HTTP
  -> React web app       questions and results; no tax logic
```

Our yardstick is **IRS Direct File**, the IRS's free filing tool, whose source code was released into the public domain. It contains a tax engine and 161 test returns. We run that engine on its own test returns and record every value it produces (83,488 rows in `oracle/facts.csv`). That gives us an independent opinion on every number.

The headline: the law as Catala

The unusual part of this project is the Catala layer. The rules are 37 Catala modules, 20,802 lines. Each section of the Internal Revenue Code or the IRS instructions is quoted, and the rule that implements it sits directly under the quote, so a tax expert who does not program can check it paragraph by paragraph. The Catala compiler type-checks all of it (`clerk typecheck`: "Typechecking successful!"), and **602 of 602** named tests pass (`make check`, 2026-11-01, per What "verified" means here). Each module has hand-worked examples from the forms and instructions, tests that a known-wrong output is rejected, and one test per Direct File discrepancy that asserts the law's value and not Direct File's.

Then the whole-return Catala scope runs on the inputs of every Direct File test return, and every output is compared with the Direct File engine, exactly, in cents. Direct File wrongly knocks out 4 of its own returns (DF-BUG-8, below), leaving 157. Result of `make difftest SCOPE=calculator`, re-run for this write-up on 2026-11-01: **157 of 157 returns and 5,360 of 5,360 values agree**. 9 values are reported apart as known divergences, where we follow the law on purpose: DF-BUG-3, DF-BUG-23, and one return whose filing status Direct File's own validity check rejects. A known divergence is never counted as a pass.

What the type checker did and did not do. It is tempting to say "the compiler found bugs in the IRS's code". It did not. Catala 1.3.0 rejects type errors (money plus a decimal), non-exhaustive matches and dependency cycles at compile time, and it forced several real design decisions: two calls of the interest module to break a cycle through AGI, a staged filing status, explicit rounding at every place a dollar amount meets a rate, exact fractions instead of floating point. But **none of the 33 disagreements with Direct File came from the type checker**. 25 came from writing each rule in Catala with the statute quoted and Direct File's XML open beside it; 7 from a review of the Direct File code before any Catala was written; 1 from running Direct File's test returns. What Catala gave us is a form precise enough to compare line by line. The full table, with how each was found, is in What "verified" means here.

Bugs in the reference implementation

Where Direct File and the law disagree, our policy (set by the human) is that **the statute wins**, and we log the difference in `docs/direct-file-bugs.md`. The log has 33 entries. The calculator follows the law in 30 of them; for 3 (DF-BUG-2, 4 and 10) it keeps Direct File's behaviour until the owner decides. Every entry is pinned by a test (`docs/quirk-decisions.md`).

Status	Count	Entries
Confirmed by running the Direct File engine	1	DF-BUG-8
Read in the engine's source (arithmetic, not a tax rule)	1	DF-BUG-5
Candidate, our code already follows the law ("fixed")	2	DF-BUG-1, DF-BUG-7
Candidate	29	the rest
Visible as a different value on a Direct File test return	8	3, 8, 13, 14, 15, 19, 23, 24

- **DF-BUG-8 (confirmed).** Direct File decides whether the Form 1099-R box 8 percentage is zero by stripping the characters `0` and `.` and checking whether anything is left. Its configuration reader turns the literal `0.` into the number 0, so the `.` is never stripped and `0.00` looks nonzero. Direct File then wrongly knocks out 4 of its own test returns. **Discovered by Arnab Ghosh.**
- **DF-BUG-5.** Exact fractions converted to dollars through 32-bit floating point (`types/Dollar.scala`).
- **Candidates**, for example: every Form 1099-MISC treated as Alaska Permanent Fund income (DF-BUG-9); a foster parent getting the child care credit through a custody release, $702 where the law gives $602 (DF-BUG-23); a health insurance policy that starts December 1 never counting December (DF-BUG-31).

A candidate is our reading, with a citation and a test, not a finding the IRS has agreed with.

Why Lean, and why not a translator

Lean is a programming language with a proof checker built in. A Lean theorem is checked by a small kernel, so you do not have to trust whoever (or whatever) wrote the proof, only the statement.

The obvious plan was to translate the Catala rules into Lean automatically. Catala 1.3.0 has no Lean backend, and nothing upstream was close (`docs/catala-to-lean.md`, section 2). Writing one was estimated at 50 to 80 hours before any proof work, and its output would look like compiler internals. So the human decided:

- **Write the calculator directly in Lean**, one Lean file per Catala module. The proofs are about the code that runs.
- **Keep Catala as the readable spec and test oracle.**
- **Link the two by differential testing**: run Direct File, Catala and Lean on the same inputs and compare every value.

So the link between "the law as written in Catala" and "the Lean program" is a test, not a proof. We say that everywhere.

Copying Catala's arithmetic exactly by hand matters. The spike measured it (`docs/catala-to-lean.md`, section 3.2): drop one cent-rounding step and single-filer tax changes on 971 of 200,000 cent values between $100,000 and $102,000; round half to even instead of half away from zero and it changes on 30,700 of 100,000 dollar values below $100,000. The Lean money library reproduces both counts as a regression test (`lean/UsTax/Money/Regression.lean`).

What is proved

The Lean project is 125 files and about 40,000 lines under `lean/UsTax`, with no Mathlib. The theorems are about the calculator's own functions. Statements are in `lean/UsTax/Comparator/Challenge.lean` and `Extra.lean`, the properties in `lean/UsTax/Spec/`, and `docs/spec-surface.md` lists every definition each statement depends on and reviews it.

The 9 main theorems (`lean/comparator.json`, namespace `UsTax.Main`), none with hypotheses beyond its arguments:

Theorem	In plain words
`tentativeTax_nonneg_monotone`	Tax on taxable income (line 16) is never negative and never decreases as income increases, for each filing status.
`tentativeTax_matches_tax_table`	For every taxable income below $100,000, line 16 equals the 2024 Tax Table, in the right column.
`line16_matches_tax_table`	The same, for line 16 inside a full Form 1040.
`tentativeTax_matches_worksheet`	From $100,000 up, line 16 equals the printed Tax Computation Worksheet.
`line16_matches_worksheet`	The same, inside a full Form 1040.
`refund_balanceDue_consistent`	Refund and amount owed are nonnegative, at most one is nonzero, and refund minus owed equals payments minus tax.
`credits_within_liability`	Credits can bring the tax to zero, not below (line 22 = max(0, line 18 − line 21)).
`totalTax_nonneg`	Total tax (line 24) is never negative.
`eitc_within_bounds`	The earned income credit is between zero and its legal maximum, and zero past the phase-out.

31 more (`lean/comparator.extra.json`, namespace `UsTax.Extra`). 23 tie modules to a plain reading of the statute for every input: the standard deduction, interest, W-2 wages and withholding, taxable Social Security, the educator and student loan adjustments, estimated payments, AGI, taxable income, total payments. 8 are about the whole return the server runs: refund or bill and credits within the tax on the whole return, the child care, elderly and saver's credits together within the tax, knockout soundness (`supported = false` exactly when a reason is listed), `outcome = refund` exactly when line 34 is positive, and **no pure internal-inconsistency result on any valid request** (HTTP/runtime exceptions are outside this theorem) (`calculateRequest_ne_fault`).

Supporting results worth naming:

- `Api.parseRequest_sound`: every request the server accepts satisfies `Request.Valid` (`lean/UsTax/Api/Spec.lean`); `parseRequest_error_status`: every rejection is a 400.
- `Proofs.rows_chain`, `Proofs.rows_cover`: the 2,062 Tax Table rows cover every cent from $0 to $100,000 with no gap or overlap. They were added in PR #132, which fixed a real gap: the first version of our table data was missing the rows for $5 to $25.

Checks on the proofs. `make comparator` and `make comparator-extra` passed on 2026-11-01 for all 9 + 31 statements (recorded when issues #65 and #66 were closed): `lake check` replays the build through the kernel, and `lake comparator` checks that the proved statements are exactly the trusted ones and use only `propext`, `Quot.sound` and `Classical.choice`. `make audit` passed on the tree merged as PR #156 (`lean/` has not changed since): the axiom step found **2,212 theorems and 6,681 definitions** using only those three axioms, with no `sorry` or `native_decide`, and the forensics step checked **7,981 declarations under `UsTax` with 0 high-severity findings** (no false or contradictory hypotheses, no conclusion that reduces to `True`). Those counts include many small lemmas; we would rather be judged on the 40 above. Each of the 40 has a satisfiable instance and at least two rejected bad outputs, 82 in all (`docs/spec-checks.md`, `make spec-checks`); for example, the Tax Table statement rejects $4,016 (the exact tax without the table's midpoint rule) for a single filer with taxable income of $35,400, where the table prints $4,019.

One statement is still pending (`lean/UsTax/Comparator/Pending.lean`): each nonrefundable credit, in Schedule 3 order, limited by the tax left after the credits before it. What is proved instead is the whole-return bound (`calculator_credits_within_liability`) and the three credits whose limits chain on line 18 (`calculator_credits_le_tax`).

What is tested

What	Result (2026-11-01)	Source
Catala whole return vs Direct File (`make difftest SCOPE=calculator`)	157/157 returns, 5,360/5,360 values; 9 known divergences; 4 knocked-out returns skipped	re-run for this write-up
Lean vs Catala vs Direct File, whole return (`make lean-difftest SCOPE=calculator`)	157 returns, 5,360 three-way agree, 0 failures; 754 more values Lean = Catala where the oracle is incomplete	PR #151
The same through the API request shape (`SCOPE=api_calculator`)	157 returns, 3,105 agree, 0 failures	PR #151
Catala named tests (`make check`)	602/602	What "verified" means here
Server (`make test-server`)	46 passed, 0 failed, 1 skipped (a 500 cannot be provoked); server = command line on 315 requests	PR #151
Request validator vs JSON schema (`make api-conformance`)	about 2,100 generated requests agree	PR #151
EIC Table golden test	1,337 rows by 8 columns, 0 differ	PR #141

Every harness has a **negative control**: `DIFFTEST_PERTURB=1` and `LEAN_DIFFTEST_PERTURB=1` change the outputs on purpose, and the run must fail. It does (for example, 6,123 of 6,123 perturbed checks fail for `SCOPE=calculator`, PR #151). A harness that cannot fail proves nothing.

How it was built

US Tax was built during the hackathon by one person and an orchestrated team of AI agents (Claude). Git history starts 2026-10-31; by 2026-11-01 there were 77 merged pull requests. One orchestrator agent split the work into issues (the roadmap is issue #1), started worker agents in separate git worktrees, re-ran the builds and difftests on each pull request before merging.

The division of labour follows `CLAUDE.md` section 8:

- **The human owns the specification and the policy**: the properties in `lean/UsTax/Spec/`, the statements Comparator checks, and the decisions (write the calculator directly in Lean; statute beats Direct File; no TLS in the Lean process; which quirks to keep).
- **Models write code and proofs.**
- **A mechanical check accepts or rejects.**

Models are good at proofs and weak at choosing what to prove; when stuck they may weaken a statement or reach for `sorry`. So the checks are layered:

1. **`make audit`** (`docs/audit.md`): a source scan for `sorry`, `admit`, `native_decide`, `axiom`, `implemented_by`, `unsafe` and similar; an axiom audit of every declaration under `UsTax`; and for-all-dev/forensics, which flags vacuous theorems (false or contradictory hypotheses, conclusions that reduce to `True`).
2. **Negative controls for the audit itself**: `scripts/audit_negative_control.sh` injects six kinds of cheat into a scratch copy (a `sorry`, an axiom, `native_decide`, `maxHeartbeats 0`, an axiom hidden in a macro, a theorem with a `False` hypothesis); each must make the audit fail (PR #116).
3. **Comparator in a Linux container** (`docs/comparator.md`). `lake check` and `lake comparator` need Linux sandboxing, so on a Mac they run in Apple's `container`. Two negative controls were run (PR #105): a `sorry` (which `lake build` alone accepted, exit 0) and an extra `axiom cheat : False` plus a changed statement. Both were rejected.
4. **Property-based testing before proving** with `plausible`. This caught a false statement: a filing-status lemma needed an extra hypothesis (PR #134).
5. **Satisfiable instances and rejected bad outputs** for every judged statement, checked by `make spec-checks`.

What went wrong, and the merge guard. Parallel agents on one repository are fast, and they can step on each other. On 2026-10-31, PR #143 was squash-merged from a branch based on an old `main`. The squash silently undid work that had been merged after that base: the provenance index, the household knockouts and their tests. It was caught, reverted and re-applied the same night. Since then every merge goes through a guard: `git diff --diff-filter=D --name-only origin/main...<branch>` must be empty (a branch may not delete files it did not mean to), and the branch must contain `origin/main`.

What we did not do. Nobody has done the full line-by-line review of the spec files that `CLAUDE.md` asks for. The theorem statements were written by a model and checked by tools (`docs/spec-surface.md` lists the "choices to confirm").

> **Spec sign-off.** The owner (Arnab Ghosh) accepted the spec files (`lean/UsTax/Spec/`, `lean/UsTax/Comparator/Challenge.lean`, `Extra.lean`, `lean/UsTax/Api/Spec.lean`) as they stand for the hackathon submission, on 2026-11-01, **without a detailed review**. Treat that as acceptance of the defaults, not as an independent audit. The open questions in `docs/quirk-decisions.md` keep their documented defaults.

Future work

- A detailed independent review of every spec file, recorded in `docs/spec-surface.md`, and decisions on the open quirk questions (`docs/quirk-decisions.md`, "Questions for the human").
- Prove the pending per-credit statement, and decide which of the whole-return theorems become main theorems.
- Report the confirmed Direct File bug upstream, and try to confirm more candidates by running the engine.
- A Catala-to-Lean translator, which would turn the testing link into a proof about a translation. Catala PR #1088 (a JSON IR export, open as of 2026-10-31) would help.
- Tax year 2025.

Numbers not re-verified for this version

Every number above was either re-run on 2026-11-01 for this write-up (`make difftest SCOPE=calculator`, `make provenance-check`, file and line counts, `lean/comparator*.json`) or copied from the source named next to it. These were **not** re-run, because building Lean was off limits while this was written:

- `make audit`: 2,212 theorems and 6,681 definitions from the axiom step on the PR #156 branch (`SKIP_FORENSICS=1`, PR #156 description); 7,981 declarations and 0 high-severity findings from the full run, forensics included, on the rebased tree before PR #156 was merged.
- `make comparator` / `make comparator-extra` PASS on all 40 statements: issue #65 and #66 closing comments.
- `make lean-difftest` (5,360 and 3,105), `make test-server` (46/0/1), `make api-conformance`: PR #151.
- 602 Catala tests: What "verified" means here. A plain count of `#[test]` markers in `catala/tests/` gives 601.

Spec surface of the main theorems (issue #63)

A theorem means what its statement says, and the statement means what every definition it mentions means, transitively (CLAUDE.md section 4). This file lists, for each main theorem, those definitions (`#spec_surface`, `lean/UsTax/Audit.lean`) and reviews them in prose.

Where things are:

what	file	in `lake build` / `make audit`
the properties (`NonnegMonotone`, `RefundOrBalanceDue`, `CreditsWithinLiability`, `EitcWithinBounds`, `MatchesTaxTable`, `worksheetTax`), each with a satisfiable example and rejected bad outputs	`lean/UsTax/Spec/*.lean`	yes
the main theorem statements, `sorry` proofs (trusted)	`lean/UsTax/Comparator/Challenge.lean`	no (comparator challenge)
the same theorems, proved; concrete instances	`lean/UsTax/Proofs/Main.lean` (solution module)	yes
lemmas they use	`lean/UsTax/Proofs/Form1040.lean`, `lean/UsTax/Proofs/TaxTable.lean`, `lean/UsTax/Proofs/TaxWorksheet.lean`, `lean/UsTax/TaxRates/Lemmas.lean`	yes
main theorems not proved yet	`lean/UsTax/Comparator/Pending.lean`	no
names checked by `lake comparator`	`lean/comparator.json`	

The `UsTax/Spec/*.lean` files import nothing from the calculator: only `FilingStatus` (`UsTax/Types.lean`) and the generated Tax Table data. The statements in `Challenge.lean` apply those properties to the calculator's outputs (`tentativeTaxFor`, `form1040`), so every statement about a Form 1040 line also has the whole calculator on its surface. That is unavoidable for a statement about the code that runs; what a reviewer has to read is the property (short) and which output fields it is applied to.

The statements

```
theorem tentativeTax_nonneg_monotone (fs : FilingStatus) :
    NonnegMonotone (fun ti => tentativeTaxFor ti fs)

theorem refund_balanceDue_consistent (i : Form1040In) :
    RefundOrBalanceDue (form1040 i).totalTax (form1040 i).totalPayments
      (form1040 i).overpayment (form1040 i).balanceDue

theorem credits_within_liability (i : Form1040In) :
    CreditsWithinLiability (form1040 i).totalTentativeTax (form1040 i).nonRefundableCredits
      (form1040 i).taxLessNonRefundableCredits

theorem totalTax_nonneg (i : Form1040In) : 0 ≤ (form1040 i).totalTax
```

```
theorem tentativeTax_matches_tax_table (fs : FilingStatus) (ti : Int) :
    MatchesTaxTable fs ti (tentativeTaxFor ti fs)

theorem line16_matches_tax_table (i : Form1040In) :
    MatchesTaxTable i.filingStatus (form1040 i).taxableIncome
      (form1040 i).tentativeTaxFromTaxableIncome

theorem tentativeTax_matches_worksheet (fs : FilingStatus) (ti : Int)
    (h1 : 100_000_00 ≤ ti) (h2 : ti ≤ 1_000_000_000_000_00) :
    tentativeTaxFor ti fs = worksheetTax fs ti

theorem line16_matches_worksheet (i : Form1040In)
    (h1 : 100_000_00 ≤ (form1040 i).taxableIncome)
    (h2 : (form1040 i).taxableIncome ≤ 1_000_000_000_000_00) :
    (form1040 i).tentativeTaxFromTaxableIncome =
      worksheetTax i.filingStatus (form1040 i).taxableIncome
theorem eitc_within_bounds (i : Eitc.EitcTableIn) :
    EitcWithinBounds (Eitc.caseOf i) (Eitc.eitc i)
```

All in namespace `UsTax.Main`. The first six and (4) have no hypothesis besides their arguments (every `FilingStatus`, every `Int`, every `Form1040In`, every `EitcTableIn`). `MatchesTaxTable` has two inside (`r ∈ rows`, `InRow r ti`); `Proofs/Main.lean` discharges both on a concrete return. (5c) and (5d) have two: line 15 is at least $100,000 (where the instructions switch from the Tax Table to the worksheet) and at most $1 trillion (the code's `noUpperBound` sentinel, above which the Catala marginal form, and so the code, stops taxing; see `TaxRates/Spec.lean`). `Proofs/Main.lean` discharges both on a concrete return (single, wages $200,000, line 15 $185,400, line 16 $37,539) and at $100,000 exactly. Together (5b) and (5d) cover every line 15 from $0 to $1 trillion: below $100,000 the printed Tax Table, from $100,000 the printed worksheet. Axioms of all nine (`#audit_axioms`): `propext`, `Classical.choice`, `Quot.sound`.

The properties (review these)

`NonnegMonotone f` (`Spec/TaxMonotone.lean`): `(∀ x, 0 ≤ f x) ∧ ∀ a b, a ≤ b → f a ≤ f b`. Not strict: a Tax Table row has one tax for all its incomes, so strict monotonicity is false. Satisfied by `max 0 x / 10`; rejects a cliff (`$100` below $10,000, `$0` above) and `x − 100`.

`RefundOrBalanceDue tax payments over owe` (`Spec/RefundBalance.lean`): `0 ≤ over ∧ 0 ≤ owe ∧ (over = 0 ∨ owe = 0) ∧ over − owe = payments − tax`. The issue wrote `over * owe = 0`; `over = 0 ∨ owe = 0` is the same over `Int` and needs no multiplication. It determines both lines (`RefundOrBalanceDue.overpayment_eq`: `over = max 0 (payments − tax)`, `owe = max 0 (tax − payments)`), so it is a full specification of lines 34 and 37, not only a sanity check. Rejects: refund and amount owed together, a refund off by one cent, a negative amount owed.

`CreditsWithinLiability before credits after` (`Spec/Credits.lean`): `after = max 0 (before − credits)`, the text of line 22. Proved consequences: credits used (`before − after`) `≤ before` and `≤ credits`, and `0 ≤ after`. Rejects: unused credit paid out as negative tax; credits ignored.

`MatchesTaxTable fs ti tax` (`Spec/TaxTable.lean`): `∀ r ∈ rows, InRow r ti → tax = dollars (printedTax r fs)`, with `InRow r ti := dollars r.atLeast ≤ ti ∧ ti < dollars r.butLessThan` (cents), `dollars d := d * 100`, and `printedTax` picking the column (qualifying surviving spouse uses the married filing jointly column, as the printed header says). Satisfied by single $50,000 ⟶ $6,059; rejects $6,053 (the tax on the exact income, without the row midpoint) and the single column used for a joint return.

`worksheetTax fs ti` (`Spec/TaxWorksheet.lean`): the 2024 Tax Computation Worksheet (Form 1040 instructions p 76; p 33 says to use it when line 15 is $100,000 or more). `worksheetRow fs ti` is the p 76 table typed from the PDF: the section of the filing status (A single, B joint and qualifying surviving spouse, C separate, D head of household), the row whose "over $X but not over $Y" contains `ti`, and its (b) multiplication amount (percent) and (d) subtraction amount (cents). `worksheetTax` is `roundHalfUp (c − d) 100 * 100` with `c := roundHalfUp (ti * rate) 100`: column (c) rounded to the cent, the tax rounded to the whole dollar (p 23, "drop amounts under 50 cents and increase amounts from 50 to 99 cents"); `roundHalfUp n d := (2n + d) / (2d)`. Choice to confirm: (c) is rounded to the cent before subtracting. This only matters for a line 15 with cents, and is what the code (Catala `money * decimal`) does. Satisfied by single $185,400 → $37,539; rejects $37,538 (rounded down), the Section B row for a single filer, 24% with nothing subtracted, and Section A for married filing separately at $400,000. Cross-check: each (d) equals `X × R − B` for the Rev. Proc. 2023-34 row "over $X: $B plus R% of the excess" (`#guard` in `Proofs/TaxWorksheet.lean`, and implied by the proof of `taxSpec_eq_worksheetTax`, which equates the worksheet with the rate schedules for every income from $100,000). `#guard`s also compare both at $100,000, every $1,000 to $2,000,000, and one cent either side of every bracket edge, for all five statuses.

`EitcWithinBounds c credit` (`Spec/Eitc.lean`, now theorem (4), proved): `0 ≤ credit ∧ credit ≤ eitcMaxCredit children ∧ (eitcCompletedPhaseout children mfj ≤ max earned agi → credit = 0)`. Amounts typed from Rev. Proc. 2023-34 § 3.06(1) (checked against `catala/params_2024.catala_en`). Rejects: $1 over the maximum, a credit at the completed phase-out, a negative credit.

`#spec_surface` output, reviewed

`#spec_surface` lists constants from modules under `UsTax`. Below, structure projections, constructors and recursors are dropped (they are fixed by the structure they belong to); counts are of the raw output.

(1) `tentativeTax_nonneg_monotone`: 37 constants, 26 definitions and types

- `UsTax.Spec.TaxMonotone`: `NonnegMonotone`
- `UsTax.TaxRates`: `Bracket`, `marginalTax`, `noUpperBound`, `pct`, `roundedTaxableIncomeOf`, `scheduleFor`, `scheduleHeadOfHousehold`, `scheduleMarriedSeparate`, `scheduleMfjQss`, `scheduleSingle`, `tentativeTaxFor`, `usd`
- `UsTax.Money`: `divMoney`, `floor`, `max`, `min`, `mulRate`, `positive`, `roundDollar`, `roundRat`, `stepwiseMultiply`, `sum`, `truncate`, `truncateRat`
- `UsTax.Types`: `FilingStatus`

Review. `NonnegMonotone` as above. `tentativeTaxFor` and everything under it is the line 16 calculation (Tax Table midpoint, then the marginal sum over the 2024 brackets, rounded to the dollar). It is the object of the theorem, not an assumption: the theorem holds for whatever the brackets are, as long as rates are nonnegative. Its agreement with the printed rate schedules is a separate theorem (`TaxRates.tentativeTaxFor_eq_taxSpec`, not in the comparator set; see "Choices to confirm").

(2) `refund_balanceDue_consistent`: 273 constants, 90 definitions and types

- `UsTax.Spec.RefundBalance`: `RefundOrBalanceDue`
- the calculator surface below

(3a) `credits_within_liability`: 272 constants, 90 definitions and types

- `UsTax.Spec.Credits`: `CreditsWithinLiability`
- the calculator surface below

(3b) `totalTax_nonneg`: 269 constants, 89 definitions and types

- no property definition; the calculator surface below

(5a) `tentativeTax_matches_tax_table`: 70 constants, 52 definitions and types

- `UsTax.Spec.TaxTable`: `InRow`, `MatchesTaxTable`, `dollars`, `printedTax`
- `UsTax.TaxRates.TaxTableData` (generated): `Row`, `rows`, `rows0` … `rows20`
- the line 16 surface of (1): `UsTax.TaxRates` and `UsTax.Money` as there, `FilingStatus`

(5b) `line16_matches_tax_table`: 304 constants, 116 definitions and types

- `UsTax.Spec.TaxTable` and `UsTax.TaxRates.TaxTableData` as in (5a)
- the calculator surface below

(5c) `tentativeTax_matches_worksheet`: 39 constants, 28 definitions and types

- `UsTax.Spec.TaxWorksheet`: `roundHalfUp`, `worksheetRow`, `worksheetTax`
- the line 16 surface of (1): `UsTax.TaxRates` and `UsTax.Money` as there, `FilingStatus`

(5d) `line16_matches_worksheet`: 273 constants, 92 definitions and types

- `UsTax.Spec.TaxWorksheet` as in (5c)
- the calculator surface below

Review. The three worksheet definitions are short and import nothing from the calculator; what to check is that `worksheetRow` matches p 76 (bounds, rates, subtraction amounts, "not over" inclusive) and the rounding.

(4) `eitc_within_bounds`: 54 constants (50 definitions and types)

- `UsTax.Spec.Eitc`: `EitcWithinBounds`, `EitcCase`, `eitcMaxCredit`, `eitcCompletedPhaseout`
- `UsTax.Eitc.Case`: `caseOf`, `eitc` (read off `EitcTableIn`, take `.amount`)
- `UsTax.Eitc`: `EitcTableIn`, `EitcTableOut`, `eitcTable`, `phaseoutReduction`, `dollars`
- `UsTax.Params.*` (`UsTax/Params2024/EitcCtc.lean`): the eight credit and phase-out percentages (`decPct`), the four maximum credits, the four thresholds, the eight completed phase-out amounts, `eitcTableRowWidth`, `eitcTableRowMidpoint`, `eitcTableRowLastDollarOffset`
- `UsTax.Money`: `divMoney`, `mulRate`, `roundRat`, `roundDollar`, `stepwiseMultiply`, `truncate`, `truncateRat`, `max`; `UsTax.TaxRates.usd`

Review. This is the whole `EitcTable` scope (the printed-table emulation), not the eligibility rules of `EarnedIncomeCredit`: the property is about the credit for four facts (children, joint or not, earned income, AGI), as #67 asks. Two choices to confirm: (a) `caseOf` reads a negative child count as 3 (the code's `else` column), so the theorem needs no hypothesis; a count is never negative in a real return. (b) The maximum credits and completed phase-out amounts are typed twice, in `Spec/Eitc.lean` from the Rev. Proc. and in `Params2024/EitcCtc.lean` from `catala/params_2024.catala_en`; the proof (not the statement) shows they agree, so a typo in either makes the theorem unprovable, not wrong. The 1,337-row printed EIC Table golden test (`UsTax/Eitc/EicTable.lean`) is what ties `eitcTable` to the IRS table; the theorem does not claim that.

The calculator surface (shared by 2, 3a, 3b, 5b, 5d)

- `UsTax.Form1040`: `Form1040In`, `Form1040Out`, `form1040`
- `UsTax.FormW2s`: `CombatPayElection`, `FormW2`, `FormW2Item`, `FormW2sIn`, `FormW2sOut`, `formW2Item`, `formW2s`, `instDecidableEqCombatPayElection`

- `UsTax.Interest`: `InterestIn`, `InterestOut`, `InterestReport`, `InterestReportFacts`, `interest`, `interestIncome`, `interestReportFacts`, `requiresScheduleB`
- `UsTax.StandardDeduction`: `StandardDeductionIn`, `StandardDeductionOut`, `additionalStandardDeduction`, `additionalStandardDeductionItems`, `additionalStandardDeductionMultiplier`, `bit`, `filerMfsAndSpouseIsEligibleForStandardDeductions`, `filingStatusStandardDeduction`, `isSpouseInfoApplicableForStandardDeductions`, `minimumStandardDeduction`, `normalStandardDeduction`, `secondaryFilerAdditionalStandardDeductionItems`, `secondaryFilerIs65OrOlderForDeduction`, `standardDeduction`, `standardDeductionAmount`
- `UsTax.TaxRates`: as in (1), plus `TentativeTaxIn`, `TentativeTaxOut`, `tentativeTax`, `cumulativeTax`, `tentativeTaxCumulativeFor`
- `UsTax.Params2024` (`UsTax.Params.*`): `additionalMedicareTaxThreshold{Joint,MarriedSeparate,Other}`, `medicareWagesIndividualLimit`, `niitThreshold{JointOrSurvivingSpouse,MarriedSeparate,Other}`, `oasdiWageBase`, `scheduleBInterestThreshold`, `stdDeduction{AdditionalMarried,AdditionalUnmarried,DependentEarnedIncomeAddition,DependentFloor,HeadOfHous`
- `UsTax.Money` as in (1), plus `mulInt`; `UsTax.Lists`: `Money.sumBy`
- `UsTax.Types`: `FilerRole`, `FilingStatus` and their `DecidableEq` instances

Review. This is the calculator up to line 37; the theorems are about its outputs, so it is on the surface but it is not assumed correct. What the theorems rely on is only which field is which line: `totalTax` is line 24, `totalPayments` line 33, `overpayment` line 34, `balanceDue` line 37, `totalTentativeTax` line 18, `nonRefundableCredits` line 21, `taxLessNonRefundableCredits` line 22, `taxableIncome` line 15, `tentativeTaxFromTaxableIncome` line 16 (field docstrings in `UsTax/Form1040.lean`). The proofs of (2), (3a), (3b) do not depend on the values of any parameter: they hold for every input, including every plug-in slot (`Form1040In` fields with default `0`) and negative ones.

The data `rows` is part of the spec of (5): it is generated from `sources/forms-instructions/derived-tax-table-2024.csv` by `scripts/gen_lean_tax_table.py` and is what "the printed Tax Table" means in Lean. A wrong CSV row makes (5) say something wrong.

Pending (not in `comparator.json`)

Templates in `lean/UsTax/Comparator/Pending.lean`:

- **(3c) Credit wiring** (#66): each nonrefundable credit, computed by its own module, satisfies `CreditsWithinLiability` against the tax left after the credits before it (Schedule 8812 Credit Limit Worksheet A, Schedule 3 order), and line 21 is their sum. Needs CTC/ODC (#27) and Schedule 3 in Lean. Today line 21 is a plug-in slot, and (3a) holds for whatever value it has.

Each moves into `Challenge.lean` and `comparator.json` when proved. (4), the EITC bounds, moved in with the Lean EITC (#46, #67).

Choices to confirm (human)

1. **(5) covers all taxable income up to $1 trillion.** Below $100,000 the Tax Table (5a, 5b); from $100,000 the Tax Computation Worksheet (5c, 5d), written from the printed worksheet, not from `taxSpec` (three definitions on the surface instead of about ten). The upper bound is the code's `noUpperBound` sentinel (from Catala); above it line 16 is wrong (proved, `TaxRates/Spec.lean`), so (5c, 5d) need it. Removing it means changing the Catala source and the code. Rounding (c) to the cent: see `worksheetTax`.
2. **The table data covers every cent below $100,000.** An earlier version of the CSV lacked the printed rows [$5, $15) ($1) and [$15, $25) ($2); fixed. `Proofs.rows_chain` (kernel) shows the 2,062 rows tile $0 to $100,000

with no gap or overlap, and `Proofs.rows_cover` that every cent below $100,000 is in a row, so (5) constrains line 16 for all of them.

3. **Credits (3a) are stated at line 22 given line 21**, not per credit. Line 21 is today an input slot; the per-credit statement is pending (3c).
4. **(3b) is `0 ≤ totalTax` with line 23 not added** (QUIRK(Q3) in `UsTax/Form1040.lean`). When line 23 is wired, (3b) still holds only if line 23 is nonnegative; the proof will need that.
5. **`RefundOrBalanceDue` uses `over = 0 ∨ owe = 0`** instead of the issue's `over * owe = 0` (equivalent over `Int`).
6. **EITC "zero at the completed phase-out" uses `max earned agi`** (IRC 32(a)(2)(B)) and ≤ (zero at exactly the completed amount). MFS uses the "other" column, as in Direct File.

Module specs (second challenge, `comparator.extra.json`)

Not main theorems: these say that each merged module of the calculator equals a short statute-style spec, for **every** input, plus bounds and monotonicity. The specs import nothing from the calculator except `FilingStatus` (and `Spec/Inputs.lean`, which reads plain amounts off the input records). Rounding is restated from scratch (`Spec/Rounding.lean`: `roundHalfAway`, `roundToDollar`, `percentOf`) and proved equal to the calculator's `Money` functions (`Proofs/Rounding.lean`).

where	what
`lean/UsTax/Spec/{Rounding,StandardDeduction,Interest,FormW2,SocialSecurity,Adjustments,Form1040Lines,Inputs}.lean`	the specs, each with examples and rejected bad outputs
`lean/UsTax/Proofs/{StandardDeduction,Interest,FormW2,SocialSecurity,Adjustments,Form1040Lines}.lean`	the proofs; each theorem on a concrete input (hypotheses discharged by `decide`) and a general one
`lean/UsTax/Proofs/SpecProps.lean`	plausible tests of every statement (run before proving; run on every build), plus a negative control
`lean/UsTax/Comparator/Extra.lean`	the 23 module statements and the 8 whole-return statements of #48 (below) with `sorry` (challenge)
`lean/UsTax/Proofs/Extra.lean`	the same, proved (solution)
`lean/comparator.extra.json`	`make comparator-extra`

The statements (namespace `UsTax.Extra`)

theorem	says
`standardDeduction_eq_spec`	line 12 = `standardDeductionSpec fs items canBeClaimed earned` (IRC 63(c): basic + items × additional; dependent: `min(basic, max($1,300, earned + $450)) + additional`)
`additionalItems_range`	`0 ≤ items ≤ 4`, and ≤ 2 unless joint or treated as MFS
`requiresScheduleB_eq_spec`	Schedule B iff taxable interest > $1,500 or a foreign account/trust
`interestIncome_eq_spec`	line 2b = Σ per-payer rounded amounts with Schedule B, else the total rounded once
`wages_eq_spec`, `formW2Withholding_eq_spec`	lines 1a, 25a = Σ of box 1 / box 2, each W-2 rounded to the dollar
`medicareKnockout_eq_spec`	knockout iff Medicare wages > $250k MFJ / $125k MFS / $200k other, or a joint spouse > $200k (IRC 3101(b)(2), 3102(f))
`oasdiKnockout_eq_spec`	per filer: knockout iff more than one W-2 and Σ min(box 3, $168,600) > $168,600

theorem	says
line6b_eq_spec	line 6b = taxableSocialSecurity (IRC 86(a)(1)/(a)(2) tiers on provisional income, base/adjusted base $25k/$34k, $32k/$44k joint, $0/$0 MFS together)
line6b_bounds	0 ≤ line 6b ≤ roundToDollar(85% × max 0 line 6a)
line6b_mono_otherIncome	more other income never lowers line 6b
educatorAdjustment_eq_spec	line 11 = Σ min($300, expenses) of each educator, rounded; $0 for MFJ dependents
educatorAdjustment_bounds	0 ≤ line 11 ≤ $300 × number of educators (nonnegative expenses)
studentLoanAdjustment_eq_spec	line 21 = round(min(interest, $2,500) × (1 − min(1, (MAGI − start)⁺ / range))), $0 if MFS or not eligible
studentLoanAdjustment_bounds	0 ≤ line 21 ≤ min(round(interest), $2,500) (nonnegative interest)
studentLoanAdjustment_zero_of_magi_ge_end	$0 from MAGI $95,000 ($195,000 joint)
studentLoanAdjustment_antitone	more total income never raises line 21 (nonnegative interest)
estimatedPayments_eq_spec	line 26 = payment rounded to the dollar, or $0
agi_eq	line 11 = line 9 − line 10
taxableIncome_eq	line 15 = max 0 (line 11 − line 14)
totalPayments_eq	line 33 = 25d + 26 + 32 when line 26 is whole dollars
totalPayments_eq_full	the same on Form1040Full, unconditionally
totalTax_mono_addW2	adding a W-2 with box 1 ≥ 0 never lowers line 24 (everything else fixed)

Every theorem is instantiated on a concrete input with each hypothesis discharged, and a known-bad output is rejected, next to the proof. Axioms of all 23: propext, Classical.choice, Quot.sound.

#spec_surface output, reviewed

Counts are of the raw #spec_surface output (constants), then of the definitions and types after dropping projections, constructors, recursors and instances. "Spec" lists the spec-side definitions (review these); the rest is the module under test, on the surface because the statement is about it.

theorem	constants	defs	spec definitions
standardDeduction_eq_spec	57	33	standardDeductionSpec, basicStandardDeduction, additionalAmountPerItem
additionalItems_range	53	30	none
requiresScheduleB_eq_spec	63	20	scheduleBRequired, reportTaxableInterest
interestIncome_eq_spec	66	23	interestLine2b, scheduleBRequired, reportTaxableInterest, roundToDollar, roundHalfAway
wages_eq_spec, formW2Withholding_eq_spec	107	24	totalOfBox, roundToDollar, roundHalfAway
medicareKnockout_eq_spec	109	26	needsForm8959, additionalMedicareThreshold, filerBox, roundToDollar, roundHalfAway
oasdiKnockout_eq_spec	110	25	excessSocialSecurityPossible, filerBox, roundToDollar, roundHalfAway

theorem	constants	defs	spec definitions
line6b_eq_spec	54	31	taxableSocialSecurity, ssTaxableTiers, ssBaseAmount, ssAdjustedBaseAmount, ssModifiedAgi, percentOf, roundToDollar, roundHalfAway
line6b_bounds	50	26	percentOf, roundToDollar, roundHalfAway
line6b_mono_otherIncome	47	23	none
educatorAdjustment_eq_spec	29	15	educatorDeduction, educatorShare, tpIsEducator, spouseIsEducator, roundToDollar, roundHalfAway
educatorAdjustment_bounds	25	11	tpIsEducator, spouseIsEducator
studentLoanAdjustment_eq_spec	41	22	studentLoanDeduction, studentLoanEligible, studentLoanPhaseoutStart, studentLoanPhaseoutRange, roundHalfAway
studentLoanAdjustment_bounds	38	19	roundToDollar, roundHalfAway
studentLoanAdjustment_zero_of_magi_ge_end	38	19	studentLoanPhaseoutStart, studentLoanPhaseoutRange
studentLoanAdjustment_antitone	37	17	none
estimatedPayments_eq_spec	23	9	estimatedPaymentsLine26, roundToDollar, roundHalfAway
agi_eq, taxableIncome_eq, totalPayments_eq	271	83	none (the calculator surface above)
totalPayments_eq_full	506	138	none (calculator + every Form1040Full module)
totalTax_mono_addW2	270	83	none

Review. Each spec definition is a few lines with its IRC / instruction citation in the docstring (`lean/UsTax/Spec/*.lean`). The module-side names (`StandardDeduction.*`, `Params.*`, `Money.*`, `SocialSecurity.*`, ...) are the object of the theorem, not assumptions: a wrong parameter in `Params2024` makes the `_eq_spec` theorem fail to prove, because the spec types the 2024 amounts itself.

Choices to confirm (human)

1. **Rounding is part of the spec.** Line 6b's bound is `≤ roundToDollar(85% × line 6a)`, not `≤ 0.85 × line 6a`: $1 of benefits gives 85 cents, line 6b $1 (the worksheet rounds to the dollar). Likewise line 21 `≤ min(round(interest), $2,500)`: $10.60 of interest deducts $11. Stated honestly rather than with whole-dollar hypotheses.
2. **Line 6b's bound uses `max 0 line6a`**: net benefits can be negative (repayments); then line 6b is `$0` and 85% of a negative amount would be below it.
3. **Who counts as a standard-deduction item** (the spouse rules) is taken from the calculator's own count, with only its range proved. The spouse rules themselves are not restated.
4. **Dependent's earned income** is line 1z (wages), as in Direct File; IRC 63(c)(5) also counts self-employment income, which is out of scope.
5. **Educator cap per educator, not per filing status**: the module never looks at the filing status; "spouse is an educator" on a non-joint return is the interview's job. The bound is `$300 × number of educators`.
6. **Student loan uses the exact fraction**, rounded once (DF-BUG(7)); Direct File's three-decimal ratio differs by up to a few dollars (the spec rejects DF's $1,668 at $85,000 MAGI).

7. **Line 33 needs line 26 in whole dollars** on the strict `form1040` (a plug-in slot that callers may set to cents); on `Form1040Full` the estimated-payments module guarantees it, so `totalPayments_eq_full` has no hypothesis.
8. **Line 24 is monotone in wages only with credits held fixed.** On `form1040` every credit is an input slot, so adding a W-2 changes no credit. Not claimed: refund (line 34) monotone, or anything with EITC or CTC computed from wages (false: more wages can lower EITC). The theorem holds for dependents too (their standard deduction grows by at most the added wages).
9. **Monotone in "other income"** for line 6b is stated by changing line 8 of the worksheet input; the proof covers any change of modified AGI (`Proofs.tiers_mono`).

The whole return and the API answer (issue #48, `comparator.extra.json`)

Eight more statements of the second challenge (`lean/UsTax/Comparator/Extra.lean`, section "The whole return and the API answer"; proofs `lean/UsTax/Proofs/Calculator.lean`, plausible tests run first in `lean/UsTax/Calculator/Props.lean`). They are about `Calculator.calculator` (the Lean port of `catala/calculator.catala_en`) and `ApiCalculator.apiCalculator` (`catala/api_calculator.catala_en` on the typed request `Api.Request`), which the server runs. Not added to `lean/comparator.json`: the human decides which of them become main theorems.

theorem	says	property (`UsTax/Spec/*.lean`)
`calculator_refund_balanceDue_consistent`	on the whole return, refund (line 34) and amount owed (line 37) are nonnegative, exclusive, and their difference is line 33 – line 24 (#65)	`RefundOrBalanceDue`
`calculator_credits_within_liability`	line 22 = `max 0 (line 18 − line 21)`, line 21 = lines 19 + 20: the nonrefundable credits used never exceed the tax (#66)	`CreditsWithinLiability`
`calculator_totalTax_nonneg`	line 24 ≥ 0	none
`calculator_credits_le_tax`	CDCC + Schedule R + Saver's ≤ line 18 of the tax pass (line 17 included): the per-credit limits of `NonrefundableCredits/Lemmas.lean` composed in the calculator's wiring (#66)	`CreditsWithinTax` (new, `Spec/Calculator.lean`)
`calculator_unsupported_iff`	`unsupported` (every module's own flag, `/flowIsKnockedOut`) holds exactly when the knockout table lists a reason	none
`apiCalculator_supported_iff`	`supported = false` exactly when `reasons ≠ []`	`KnockoutSound` (new)
`apiCalculator_outcome_refund_iff`	`outcome = refund` exactly when `lines.overpayment > 0` (an unsupported return: `even`, lines zero)	`RefundOutcome` (new)
`calculateRequest_ne_fault`	`Api.calculateRequest` never returns its internal-inconsistency result `fault` on a valid request; HTTP/runtime exceptions are outside this theorem	none

None has a hypothesis except `calculateRequest_ne_fault` (`r.Valid`, which `Api.parseRequest_sound` gives every parsed request); each is instantiated on a return of the Catala tests in `Proofs/Calculator.lean`. The new properties each have a satisfiable example and rejected bad outputs (`Spec/Calculator.lean`: a 422 with no

reason, a supported return with a stray reason, `refund` with nothing overpaid, each credit limited by the full tax). Axioms of all eight: `propext`, `Classical.choice`, `Quot.sound`.

`#spec_surface` (constants, then `UsTax` declarations including fields and constructors):

theorem	constants	UsTax	spec definitions
`calculator_refund_balanceDue_consistent`	3,096	2,403	`RefundOrBalanceDue`
`calculator_credits_within_liability`	3,095	2,402	`CreditsWithinLiability`
`calculator_totalTax_nonneg`	3,092	2,399	none
`calculator_credits_le_tax`	3,096	2,403	`CreditsWithinTax`
`calculator_unsupported_iff`	3,093	2,400	none
`apiCalculator_supported_iff`	3,841	3,105	`KnockoutSound`
`apiCalculator_outcome_refund_iff`	3,842	3,106	`RefundOutcome`
`calculateRequest_ne_fault`	3,966	3,204	none

Review. The surface is the whole calculator (every merged module, `KnockoutsIncome`, `KnockoutsHousehold`, `Calculator`: about 485 of the `UsTax` names) and, for the API statements, the typed request and its mapping (`Api.Types`, `Api.Bridge.toCalculatorInput`, `ApiCalculator`) and, for `calculateRequest_ne_fault`, the JSON reader and validator. That is the code that runs; what a reviewer reads is the property (a line each) and which outputs it is applied to. Choices to confirm:

1. `calculator_credits_le_tax` is about the three credits whose limits chain on line 18 (Form 2441 line 10, Schedule R line 3, Form 8880 line 11). The child tax credit (Schedule 8812 Credit Limit Worksheet A) is rounded to the dollar after its limit, so "all four ≤ line 18" holds only to the dollar; it is not claimed. What the return guarantees for all credits is `calculator_credits_within_liability` (line 22 is floored at zero).
2. `calculator_unsupported_iff` rests on each module's `unsupported` being the disjunction of the `ko_` outputs the knockout tables list (true by definition, checked by `rfl` in the proof). A module knockout that is not passed to `KnockoutsIncome` / `KnockoutsHousehold` would make the theorem fail to prove: that is the point of it.

Catala to Lean: strategy and trust story (spike, issue #8)

Status: decision proposal, 2026-10-31. Toolchain used: Catala 1.3.0 (`clerk`), Lean 4.35.0 (core only), OCaml 5 via opam `default` switch.

1. Recommendation

Option (c): write the calculator directly in Lean. Catala stays the readable spec, written next to the law, and acts as the test oracle. A three-way differential test (Catala-generated OCaml, Lean, Direct File oracle CSV) closes the gap between them. The Part 3 server calls the Lean calculator, so the proofs are about the code that runs.

Use a refinement proof where it is cheap. On the pilot scope the spike proved that a line-by-line port of the Catala-generated code equals the readable Lean function for every input (§4.3). That pattern can be reused for a few scopes. It is not required for every module.

Do not choose option (a), a hand port of the generated code: it has the costs of (b) and (c) together and the assurance of neither. Keep option (b), a Catala to Lean translator written as a Catala backend plugin, as a funded stretch goal, not the baseline (§4.2).

Decisions the human must make (the orchestrator should raise these):

1. **(c) baseline versus (b) translator.** (c) is about 40 to 85 h for about 15 modules. (b) is about 50 to 80 h before any proof work, and its output has the shape of compiler IR, so proofs need a readable layer anyway (§5). Choosing (c) means the rules.md verification answer is "proofs about the Lean code that runs; its agreement with the Catala spec is established by exhaustive and scenario differential testing, not by proof". Choosing (b) upgrades that to "proofs about a static translation of the Catala code" at the cost of an unverified translator of about 1,000 lines of OCaml.
2. **What the Part 3 server runs.** The recommendation assumes the server calls the compiled Lean calculator. If the server runs the Catala-generated OCaml or Python instead, the Lean proofs become proofs about a model, and the rules.md verification score drops. This is a Part 3 decision that follows from this one.
3. **Treatment of Catala conflicts and empty definitions in Lean** (§3.4). Catala raises `Conflict` or `NoValue` at run time. A Lean `if` chain silently picks one branch. The proposal: every module states, for each exception group with overlapping conditions, either a disjointness lemma or uses the `handleExceptions` combinator. The human should confirm this is the policy.

2. Upstream status (option d)

- **No Lean backend exists in Catala 1.3.0** (backends: c, java, ocaml, python; IR dumps: scopelang, dcalc, lcalc, scalc). A search of `CatalaLang/catala` issues, PRs and branches on 2026-10-31 found no Lean backend work.
- Related upstream items:
- Open PR CatalaLang/catala#1088 (external contributor, not merged; a maintainer said on the PR that they want to discuss it first) adds `catala dcalc --output-format=json`. Its stated motivation is "differential testing against external oracles (e.g. formal ports such as a Lean 4 runtime)". If it merges, it would give option (b) a stable machine-readable IR. It is not in 1.3.0.

- Branch `coq_backend` (last commit 2024-01-08, never merged): `compiler/dcalc/to_coq.ml`, 526 lines, prints dcalc as a deep embedding (Coq terms of a default-calculus AST). It shows the size of a printer, but it is stale against the 1.3.0 AST.
- Catala's own formal work (the 2021 paper) proves the dcalc to lcalc compilation correct in F*, about 3,500 lines. It is not Lean.
- arnavdandu/catala-lean and its fork ludovicc/catala-lean: a Lean (Mathlib) formalization of a core default calculus with only integers, booleans and pairs, plus a small-step semantics and a Python differential harness. It has no money, decimal, structs or enums, and it cannot run our programs. It is not usable here.
- **Conclusion:** nothing upstream reduces the cost of (a), (b) or (c) within this project's time frame. Re-check PR #1088 before starting (b).

3. Catala runtime semantics Lean must replicate

Verified by reading `sources/reference-catala/catala/runtimes/ocaml/catala_runtime.ml` (1.3.0) and `stdlib/`, and by running the runtime: `docs/spike/ocaml/rt.ml` prints primitive results, and the Lean `#guard`s in `docs/spike/lean/Spike/Runtime.lean` reproduce them.

3.1 Numbers

Catala	Runtime representation	Lean (core)
`money`	`Z.t`, an integer number of **cents**, unbounded	`Int` cents
`decimal`	`Q.t`, an **exact arbitrary-precision rational** (Zarith)	`Rat` (in core since about 4.2x, so no Mathlib or Batteries needed)
`integer`	`Z.t`, unbounded	`Int`

There is no overflow and no floating point anywhere in the calculation path. Floats appear only in printing and in `decimal_of_float`, which we must never use.

3.2 Rounding (all half away from zero)

- `round q = sgn(q) * floor(|q| + 1/2)`, implemented as `fdiv (2n + d) (2d)` on `|q|`. Checked: `round(5/2) = 3`, `round(−1/2) = −1`, `round(−5/2) = −3`.
- `money * decimal` (`o_mult_mon_rat`): exact product in cents, then `round` to the **nearest cent**. Checked: 1 cent * 1/2 = 1 cent; -1 cent * 1/2 = -1 cent. Every `money * rate` in a Catala source is therefore a cent-rounding point.
- `money * integer`: goes through `decimal`, so it is exact.
- `money / decimal`, `money / integer`: multiply by the inverse, then round to the cent ($1.00 / 3 = 33 cents). `money / money` returns a **decimal** with no rounding. Division by zero is the `DivisionByZero` error.
- `round of money` (`money_round`): to the nearest **dollar**, half away from zero ($12.50 gives $13; -$12.50 gives -$13; -$12.49 gives -$12).
- `integer of decimal` (`Q.to_bigint`): **truncates toward zero** (-7/2 gives -3), unlike `round`. It is used for "floor to a multiple of $50" when the value is positive.
- `money of decimal`: rounds to the nearest cent, half away from zero.
- Stdlib `Money.*` is Catala code compiled with the program: `min`, `max`, `positive` (floor at $0), `truncate`, `round_by_excess`, `round_by_default`, `in_excess`, `in_default`, `sum` (a fold from $0). `Money.round_to_decimal` is an OCaml **external** (`stdlib/ocaml/money_internal.ml`). For `n <= 0` it computes `money_round (m / 10^−n) * 10^−n` with `Z.div` (truncation) on cents. It is exact for the "round to the next $1,000" pattern in CATALA_PATTERNS.md §1, but a Lean port must copy it literally, not "round to 10^k".
- The Lean model in the spike computes `round` with Lean's `/` (Euclidean division) instead of `Int.fdiv`. The divisor `2 * den` is positive, so the two agree, and `omega` understands `/` but not `fdiv` (found while proving; §4.3).

Consequences the spike measured (`docs/spike/lean/Probe.lean`):

- If the per-slice cent rounding is dropped (exact sum, then round to the dollar), single-filer tax differs on **971 of 200,000** cent values in $100,000 to $102,000.
- If half-to-even replaces half-away-from-zero at the dollar step, single-filer tax differs on **30,700 of 100,000** dollar values below $100,000. Tax Table midpoints often produce exactly $x.50.

Rounding placement is therefore not a detail. The Lean code must round where Catala rounds, and the differential test catches it when it does not.

3.3 Lists

`list of T` is an OCaml `array`. The operations are `map`, `map2` (lists of different length raise `NotSameLength`), `filter`, `fold` (`combine all ... initially`), `reduce` (an empty list gives `Absent`, which the caller handles or turns into an error), `find` (first match), `sort_asc` and `sort_desc` (stable), `length`, `concat`, and `ArrayAccess`. In Lean, `List` with `map`, `foldl`, `filter`, `find?` and `mergeSort` (stable) is enough. Note that `Money.sum` folds from the left starting at $0. Order does not matter for `Int`, but it does for the per-element cent rounding, which happens inside the `map`.

3.4 Default logic: exceptions, conflicts, empty

- Each scope variable compiles to an `Optional` (`Absent | Present (value, position)`). An exception group is evaluated by `handle_exceptions`: no exception present means the base rule applies, if its condition holds. Exactly one present means it wins. **Two or more present raise `Conflict`** at run time.
- A chain of single exceptions (`label a ... exception a` nested), as in the spike's Tax Table key, compiles to nested `match ... Absent -> next level`. No conflict is possible, so it is exactly an `if/else if` chain in priority order (seen in `out/rate.ml` and `rate.scalc`).
- A variable with no applicable rule raises `NoValue`. Assertions raise `AssertionFailed`. A `condition` variable defaults to `false`.
- In Lean under (c): write the priority chain as `if` and state as a lemma that conditions in the same group are disjoint, or use the spike's `handleExceptions` (`Except Error (Option α)`) where the Catala source really has sibling exceptions. Totality: a Lean function always returns a value where Catala would raise `NoValue`, so the differential driver must record Catala errors as outcomes (not crash), and the comparison treats "Catala error" as a mismatch to investigate.

3.5 Other

- Dates and durations (`dates_calc`) matter for ages (65 or older, EITC 25 to 64, born-on-January-1 rules). If modules use `date` arithmetic, Lean needs a port of the parts used (`add_dates` with rounding mode, `sub_dates`, `compare_periods`, which errors on ambiguous month or day comparison). That costs about 4 to 8 h if needed. Preferred: compute ages as integers at the input boundary, so dates stay out of the proved core.
- Positions (`code_location`), trace and JSON embedding (`Value.*`) are not semantics. Drop them.

4. Options, prototyped

The toy is `docs/spike/catala/spike_rate.catala_en`: the TY2024 rate schedule for all five filing statuses, with the Tax Table midpoint key from scope.md §2.1. It exercises an enum `match`, a `list of Bracket` literal, `map each ... among ... to` with `Money.sum`, `money * decimal` cent rounding, `round of` dollar rounding, `integer of (money / money)` truncation, and a three-level `label`/`exception` tree. The four `#[test]` assertions pass with `clerk test`; the Single $50,000 case gives $6,059, which matches the 2024 Tax Table.

Generated artifact sizes for this 90-line Catala file: OCaml 348 lines (plus a 49-line `.mli`), Python 229, Java 357, C 550 (plus a 59-line `.h`), scalc 145, lcalc 147, dcalc 188.

4.1 Option (a): hand-port the generated OCaml or scalc to Lean

Prototype: `docs/spike/lean/Spike/RateA.lean` (91 lines), a line-by-line port of `out/rate.ml`, built on `Spike/Runtime.lean` (93 lines: `round`, `mulMonRat`, `divMonMon`, `intOfRat`, `roundMoney`, `Money.*`, `handleExceptions`, `default`, `errorOnEmpty`). Effort: about 30 minutes for the toy, plus about 1 h for the runtime.

- **Shape.** The code follows the IR: an `Option` per exception level, `Except Error` for `DivisionByZero` and `NoValue`, inlined bracket literals. The scalc dump is the easiest IR to read for porting (imperative, but every default is explicit). The OCaml output is the easiest to port mechanically (it is already functional).
- **Cost for about 15 modules.** Runtime 10 to 12 h. Each real module generates about 500 to 2,000 lines of OCaml, so a careful port is about 4 to 8 h per module: **70 to 130 h**, repeated for every Catala change. Every human transcription is a new trusted step that cannot be regenerated.
- **Trusted.** The person or model doing the port, the Lean runtime model, the Catala front end, and the OCaml backend (only as the test oracle).
- **Verdict.** Not recommended. It is the worst trade: IR-shaped code that is hard to prove things about, and an unrepeatable manual translation.

4.2 Option (b): a Catala to Lean translator over lcalc (or scalc)

Prototype: `docs/spike/plugin/lean_spike.ml` (about 100 lines), a **Catala backend plugin** built with `dune` against the installed `catala.driver` library and loaded with `catala lean-spike --plugin-dir=...`. It calls `Driver.Passes.lcalc`, emits Lean `inductive` and `structure` declarations for every enum and struct, and prints a histogram of the lcalc nodes the toy uses. It built and ran on the first attempt after two small API fixes. So the plugin route is real and needs no Catala fork. Build and run it from a copy outside the repo, because dune writes `_build/` next to the sources.

What a full translator has to handle (from the histogram and `compiler/lcalc/to_ocaml.ml`, 1,032 lines, the closest model):

- **Expressions:** `ELit`, `EVar`, `EAbs`/`EApp` (lets are `EApp (EAbs ...)`), `EStruct`/`EStructAccess`, `EInj`/`EMatch`, `EIfThenElse`, `EArray`, `ETuple`/`ETupleAccess`, `EExternal` (cross-module references), `EAppOp` (about 60 runtime operators such as `o_mult_mon_rat`, `o_map`, `o_toint_rat`), and `EFatalError_pos`.
- **Errors.** OCaml uses `raise`. Lean needs a monadic translation (everything in `Except Error`), or `panic!`, which is unprovable. This is the largest design item and makes the output harder to prove things about.
- **Names.** Catala identifiers can collide with Lean keywords (the stdlib struct `Period` has a field `end`, which needs «end»). The `Optional` enum is polymorphic and needs a type parameter. Modules map to Lean namespaces.
- **Stdlib.** `Money_en`, `Date_en` and the others are Catala code and translate with the program. The OCaml externals (`Money_internal`, `Decimal_internal`, `Date_internal`, `List_internal`, `Period_internal`) must be hand-written in Lean.
- **Representation.** lcalc dumps are pretty-printed text, not machine-readable, so (b) has to be a plugin over the typed AST, unless PR #1088 (dcalc JSON) lands.

Cost for about 15 modules: translator plugin 30 to 50 h, Lean runtime 12 to 16 h (plus 4 to 8 h for dates if needed), about 0.5 to 1 h per module to regenerate and fix: **50 to 80 h before any proof**. Proofs then face the IR shape (see §4.3). In practice you write the readable Lean anyway and prove a refinement. So **(b) for proof purposes costs (c) plus 50 to 80 h plus 2 to 6 h per module of refinement proof**.

- **Trusted.** The translator (unverified OCaml, about 1,000 lines), Catala's front end and its passes down to lcalc (proved correct on paper in F* for the core calculus, not for the 1.3.0 implementation), and the hand-written

Lean runtime. The differential test against Catala's own OCaml backend checks the translator plus runtime on the tested inputs. That OCaml backend shares the front end, so the test does not check the front end.

- **Version risk.** The plugin uses internal compiler APIs (GADT ASTs, `Driver.Passes` signatures), which change between Catala releases. Pin Catala 1.3.0 (see `catala.opam.locked`).

4.3 Option (c): the calculator written in Lean, with Catala as spec and oracle

Prototype: `docs/spike/lean/Spike/RateC.lean` (70 lines). It has total functions, no `Option` or `Except`, the exception tree flattened into an `if` chain, and it reuses the runtime's `mulMonRat` and `roundMoney` so money semantics match Catala exactly. Effort: about 20 minutes. The two lemmas `tableKey_nonneg` and `tableKey_mono` (the "monotone bucket function" step of issue #64) close with `unfold; repeat' split; all_goals omega`.

Refinement probe (`docs/spike/lean/Refine.lean`, 121 lines, about 1.5 h): `theorem rateA_eq_rateC : RateA.tax fs ti = some (RateC.tax (fsMap fs) ti)` holds for **all** five filing statuses and **all** integer cent inputs. The IR-shaped port never errors, and it equals the readable function. `#print axioms` shows only `propext`, `Classical.choice` and `Quot.sound`. Reusable bridge lemmas, proved in core with no Mathlib: `intOfRat ((a:Rat)/(c:Rat)) = a / c` for `a, c > 0`, `round (z : Rat) = z`, and `ratLit` (rational literal equality through `divInt`).

Friction found (feeds #39, #64):

- `decide` cannot prove `(12 : Rat) / 100 = 3/25`: kernel reduction gets stuck on `Rat.inv`/`Rat.mul` normalization. Use the `ratLit` lemma (cross-multiply on `Int`). Never use `native_decide`: it adds an axiom that Comparator will reject.
- `omega` ignores hypotheses stated on `abbrev Money := Int`. Lemmas about money must be stated on `Int`, or `Money` must be a `structure` with its own lemma set. Decide this in #39.
- `omega` does not understand `Int.fdiv`. Model OCaml's `Z.fdiv` by positive divisors as `/`.

Cost for about 15 modules: runtime (money, rounding, list helpers, conflict combinator) 6 to 10 h, then 2 to 5 h per module to write from the Catala source with `#guard` copies of the Catala unit tests: **about 40 to 85 h**. Each optional refinement proof against an IR-shaped port costs 2 to 6 h per module and needs that port written first (option (a) for that module).

- **Trusted.** Only the Lean kernel, and the statements and definitions on the spec surface (CLAUDE.md §4). The Catala-to-Lean agreement is **not** proved. It is established by testing (§6). Catala, its OCaml backend and the Direct File oracle become test oracles, not trusted components of the shipped calculator.

4.4 Differential test: run on the toy

- OCaml side: `docs/spike/ocaml/driver.ml`, linked with `ocamlfind` against `catala.runtime_ocaml`, the clerk-built stdlib objects in `_build/libcatala/ocaml/*.cmx` (link order: List before Date) and the generated module. It prints `status,ti_cents,tax_cents`.
- Lean side: `docs/spike/lean/Main.lean` prints the same CSV for `RateA` or `RateC`. Compare with `cmp`.
- **Results (all byte-identical: OCaml = Lean (a) = Lean (c)):**
- every cent from -$5 to $10,000, five statuses: 5,002,505 rows;
- every dollar from -$5 to $1,000,000, five statuses: 5,000,030 rows (the issue #41 acceptance range);
- every 7 cents from -$5 to $150,000, five statuses: 10,714,645 rows. This range covers the bracket edges where sub-cent products appear.
- Speed: OCaml about 2 s and Lean (`Rat`) about 8 s per 5 M rows. Exhaustive sweeps of single-variable scopes are cheap. Multi-input scopes need scenario and random sampling instead.

5. Comparison

	(a) hand port of IR	(b) translator plugin	(c) Lean-native, Catala as oracle
Up-front	runtime 10-12 h	plugin 30-50 h + runtime 12-24 h	runtime 6-10 h
Per module (about 15)	4-8 h, redone on every Catala change	0.5-1 h regenerate	2-5 h write + tests
Total before proofs	70-130 h	50-80 h	40-85 h
Code shape for proofs	IR (Option, Except, inlined literals): poor	IR: poor, so a readable layer and refinement are needed	readable, total: good (`omega` closes the monotone key in 3 lines)
Extra trusted base	the hand transcription	translator (about 1,000 lines OCaml) + Catala passes + Lean runtime	none beyond the kernel and the spec surface
Catala-Lean link	test only	static translation + test	test only (or optional refinement proof per scope)
rules.md verification axis	"hand-written model of the code"	"static translation of the code" (best for Catala-as-code)	"proofs about the code itself" if the server runs Lean; Catala is the spec
Comparator	generated or ported defs in the challenge	generated defs in the challenge; regenerating changes the challenge	Lean defs in the challenge; stable
Main risk	silent transcription bugs, drift	schedule (largest up-front cost), Catala API churn	Lean and Catala drift, caught only by tests

6. How the differential test closes the gap (option c)

1. **Lean vs Catala** (`make lean-difftest`, #42): the same inputs go through the Catala-generated OCaml (built by `clerk`, linked into a small OCaml driver as in the spike) and the Lean executable. All outputs are compared, and Catala run-time errors (`NoValue`, `Conflict`, ...) are recorded as outcomes. Single-input scopes (rate schedule, EITC tables) are swept exhaustively at cent or dollar resolution. Multi-input scopes are covered by every oracle scenario plus random inputs from a generator, not a filter (CLAUDE.md §5).
2. **Catala vs Direct File** (#6, #7): the oracle CSV. Combined with step 1, this gives the three-way agreement Part 2 already plans.
3. **What it does not close.** Inputs that are neither enumerated nor sampled. For the rate schedule, the spike shows the residual can be closed by proof (`rateA_eq_rateC`). For other modules the residual is what remains of the trust in the test-based link, and the README must say so.
4. **Sensitivity.** The spike's rounding variants (§3.2) would have been caught (thousands of mismatches each). The flattened exception chain would not be caught if a conflict-raising input is never generated, which is why §1 item 3 asks for disjointness lemmas.

7. Comparator and the audit (#70)

- `lake comparator` exists in the pinned Lean 4.35.0 (checked with `lake comparator --help`). It reads `comparator.json` (`challenge_module`, `solution_module`, `theorem_names`, `permitted_axioms`, optional `definition_names` holes), checks that the named solution theorems prove exactly the challenge statements, that they use only permitted axioms, and that the kernel accepts them. It sandboxes the build with **`bwrap` (bubblewrap), so it is Linux only**. On this macOS machine it cannot run, so #70 needs a Linux CI job or VM.

- Under (c), the challenge module imports the calculator definitions and the spec (`Spec.lean`, #63) and states the main theorems with `sorry`. The solution proves them. The calculator definitions are therefore part of the trusted statement surface: any change to calculator code changes the challenge. This is intended. List them with `#spec_surface`.
- Permitted axioms: `propext`, `Quot.sound`, `Classical.choice`. That rules out `native_decide` (`Lean.ofReduceBool`), which the spike showed is unnecessary for `Rat` literals.

8. Required changes to Part 2 issues

- **#9 (Lean setup):** core only. `Rat`, `Int`, `omega` and `List.mergeSort` are in core 4.35; no Batteries or Mathlib was needed for the spike, including the refinement proof. Pin `leanprover/lean4:v4.35.0` (it has `lake comparator` and `lake check`).
- **#39 (Money library):** rename `roundDollar (half_up)` to **half away from zero** (it matches Catala; it equals Direct File `HALF_UP` on all signs). Add `mulRate : Int → Rat → Int` (exact product, round to the cent: **the** Catala rounding point), `divRate`, `intOfRat` (truncation toward zero), `Money.truncate`, `round_by_excess`, `round_by_default`, and a literal port of `Money.round_to_decimal`. Add `#guard`s copied from `docs/spike/ocaml/rt.ml` output. Decide `Money` as plain `Int` versus `structure` (an `abbrev` blocks `omega`). Include the `ratLit`, `intOfRat_div` and `round_intCast` lemmas from `docs/spike/lean/Refine.lean`.
- **#40 (Runtime):** shrinks to small (S). Under (c) there is no `Optional`, positions or closure plumbing: only list helpers, `handleExceptions` for real sibling-exception groups, and (if needed) the date subset. Acceptance item "Default-calculus conflict/empty cases have a documented Lean behavior" stays, as the policy in §1 item 3.
- **#41 (Pilot):** output is `lean/Ustax/Rate.lean` written directly from the Catala source (not a port of generated code). The spike already shows the 0 to 1,000,000 sweep is cheap (about 8 s). Add, as optional, the refinement theorem against a mechanical port, reusing `docs/spike/lean/Refine.lean`. `docs/porting.md` describes "write from the Catala source" plus rounding placement.
- **#42 (Lean difftest):** the OCaml side links the clerk-built stdlib `.cmx` files (see `docs/spike/ocaml/driver.ml`). It must record Catala errors as outcomes. The Lean side is a `lean_exe` that prints CSV (the spike's `Main.lean`).
- **#43 to #48 (module ports):** "Port" means "write in Lean from the Catala module". Each adds acceptance: (1) every Catala `#[test]` as a `#guard`; (2) for each exception group with overlapping conditions, a disjointness lemma or use of `handleExceptions`; (3) money operations placed where Catala rounds. Sizes hold at M. Retitle from "Port ..." to "Lean implementation of ..." if desired.
- **#63 (Spec surface):** the calculator definitions are on the surface (they are the code being proved); `#spec_surface` output must include them.
- **#64:** the monotone bucket lemma (`tableKey_mono`) is already shown cheap. The remaining work is monotonicity of the bracket sum with per-slice cent rounding (each slice `mulMonRat (positive (min x u − l)) r` is monotone in `x`, and `roundMoney` is monotone).
- **#70 (Comparator):** needs a Linux runner (bwrap). The challenge module imports calculator definitions. Forbid `native_decide`.
- **README (trust list, CLAUDE.md §7):** the Lean kernel; the spec surface; for the Catala link, "differential testing against Catala 1.3.0's OCaml backend, plus a refinement proof for the rate schedule". If (b) is later adopted, add the translator plugin and the Catala front end.
- **New optional issue, if the human wants (b):** "Catala to Lean translator plugin (lcalc)", about 50 to 80 h. It depends on #40. Start from `docs/spike/plugin/lean_spike.ml` and `compiler/lcalc/to_ocaml.ml`, and re-check upstream PR #1088 first.

9. Reproducing the spike

Run everything in a scratch directory outside the repo (CLAUDE.md §3; `dune` and `lake` write build directories). Commands assume `export PATH=$HOME/.opam/default/bin:$PATH OPAM_SWITCH_PREFIX=$HOME/.opam/default`.

1. Copy `docs/spike/catala/` to `<scratch>/catala/`, then run `clerk test`. Generate code with `catala ocaml catala/spike_rate.catala_en --stdlib=_build/libcatala -o out/spike_rate.ml`; `catala scalc|lcalc|dcalc ...` print the IRs.
2. OCaml driver: compile `docs/spike/ocaml/driver.ml` with `ocamlfind ocamlopt -package catala.runtime_ocaml,zarith -linkpkg -I _build/libcatala/ocaml <stdlib .cmx in dependency order: List_internal List_en Date_internal Date_en Duration_en MonthYear_en Period_internal Period_en Money_internal Money_en Integer_en Decimal_internal Decimal_en Stdlib_en> Spike_rate.mli Spike_rate.ml driver.ml`. Then run `./driver STEP_CENTS MAX_CENTS`.
3. Lean: copy `docs/spike/lean/` to scratch, run `lake build`, then `.lake/build/bin/spike a|c STEP MAX`. Run `lake env lean Refine.lean` (the refinement proof and `#print axioms`) and `lake env lean Probe.lean` (rounding-variant counts).
4. Plugin: copy `docs/spike/plugin/` to scratch, run `dune build ./lean_spike.cmxs`, then `catala lean-spike --plugin-dir=_build/default <file> --stdlib=... -o -`.

Direct File discrepancies found by the formalization

Policy: statute and IRS instructions win over the Direct File XML (catala/CONVENTIONS.md §8). Each entry links the XML, the law, and what we do. Status: `candidate` (suspected, needs confirmation), `confirmed` (read in XML and law, reproduced in a test), `fixed` (our code follows the law, difftest lists the scenarios that diverge).

Credit: discovered by the ustax formal verification project (Arnab Ghosh).

ID	Status	Where (DF XML)	What DF does	What the law says	Our behaviour	Owner
DF-BUG-1	fixed	`form1099Rs.xml` l.276, `constants.xml` l.915	Strips characters in `247BGH` from the 1099-R code, then whitelists: `7D`, `7M` etc. are accepted as `7`; code `1` strips to empty and is knocked out	Distribution code meaning differs per code (early distribution, disability, etc.); `7D` is not the same as `7`	Treat the code as a whole; knock out unsupported combinations	#22, #60
DF-BUG-2	candidate	`taxCalculations.xml` l.1473	American opportunity credit hard-coded `$0` with no knockout	IRC 25A: eligible filers get the credit	Knock out (Form 8863 unsupported) instead of silently under-crediting	#37, #60
DF-BUG-3	candidate	`familyAndHouseholdInfo.xml`	No qualifying-child tie-breaker	IRC 152(c)(4)	Implement tie-breaker or knock out when two claimants	#16, #60
DF-BUG-4	candidate	* per-item rounding (scope.md §3.1)	Rounds each item then sums	1040 instructions p 23: round only the total	Decide in #60 after measuring oracle impact	#60
DF-BUG-5	confirmed (engine)	`types/Dollar.scala`	Rational to Dollar via 32-bit Float	exact arithmetic	exact	#60
DF-BUG-6	candidate	`taxCalculations.xml` l.1388-1410	`line210fSchedule2` defined but not added into `totalTax`	Form 1040 line 24 = line 22 + line 23	We add line 23 (value 0 today)	#14, #60
DF-BUG-7	fixed (candidate vs. instructions)	`studentLoanAdjustment.xml`	Phase-out ratio rounded to 3 decimals, then product and quotient rounded half-even to the cent	IRC 221(b)(2)(B): exact ratio. Form instructions allow "at least three places"	Exact ratio, rounded once to the dollar. Differs from DF in ~1/3 of phase-out cases, by up to $2; no oracle scenario is in the phase-out range	#24
DF-BUG-8	confirmed	`form1099Rs.xml` (box 8 percentage knockout)	Tests "box 8 percentage is zero" with `StripChars(text, "0.")`; the XML reader turns the literal `0.` into the number 0, so `0.00` loses its dots and looks nonzero.	A zero percentage is no knockout	Zero percentage = no knockout; `unsupported` false on those 4	#22

ID	Status	Where (DF XML)	What DF does	What the law says	Our behaviour	Owner
			Result: `knockoutForForm1099RBox8Percentage` fires wrongly on 4 oracle scenarios (`1099r-box-2b-total-distribution`, `ats-2-1099r`, `ats-2-1099r-blank-taxable`, `ats-2-1099r-nonzero-taxable`); verified by running DF engine			
DF-BUG-9	candidate	`form1099Misc.xml`	`isAlaskaPfd` is always true: every 1099-MISC is treated as Alaska PFD income on Schedule 1 line 8g	Line 8g is only for the Alaska PFD	`is_alaska_pfd` input; only PFD forms summed; any other 1099-MISC is a knockout	#21
DF-BUG-10	candidate	`unemployment.xml`	In-year repayment capped at box 1 per form	1040 instructions have no such cap	Kept the cap (issue requires; only matters on inconsistent input). Human decision	#21
DF-BUG-11	candidate	`familyAndHousehold.xml` (`ageTest` / `disabilityStatusMayAffectBenefits`)	Disabled person under 18 born the same day as the oldest filer (e.g. a twin) fails the age test	IRC 152(c)(3)(B): totally and permanently disabled passes	Follow the statute; no oracle scenario hits it	#16
DF-BUG-12	candidate	`hsa.xml` (Form 8889 line 6)	With two HSAs and family coverage, splits the family limit by what each spouse contributed, assuming agreement	IRC 223(b)(5)(B)(ii) / Form 8889 line 6: split equally unless agreed otherwise. Example: $6,000 and $1,000 contributed gives $5,150 in total by law, DF gives $7,000	Equal split; no oracle scenario affected	#35
DF-BUG-13	candidate	`elderlyAndDisabled.xml` (Schedule R line 13b)	Adds 1040 line 5a minus 5b (cost recovery, direct rollovers) to line 13b	IRC 22(c)(3)(A) and Schedule R instructions p R-5 exclude those amounts	Use only the self-reported amount; 33 checks in 21 oracle scenarios (all 1099-R) diverge, none changes totalEdc	#30
DF-BUG-14	candidate	`saversCredits.xml` (Form 8880 line 4)	Sets distributions to $0 for a spouse who is a student, under 18 or	IRC 25B(d)(2)(D) / Form 8880 line 4: both spouses'	Follow the statute; 2 oracle scenarios diverge, none changes the credit	#29

ID	Status	Where (DF XML)	What DF does	What the law says	Our behaviour	Owner
			claimed as a dependent, dropping them from the other spouse's column too	amounts apply in both columns		
DF-BUG-15	candidate	`familyAndHousehold.xml` (qualifying relative)	Counts the filer's own qualifying child as a qualifying relative when the child also passes those tests	IRC 152(d)(1)(D): a qualifying relative is "not a qualifying child of such taxpayer"	Follow the statute; 2 oracle persons (`hoh-cdcc-combat-pay`, `hoh-cdcc-multiple-careproviders`); eligible-dependent status unchanged; #19 to confirm HoH unaffected	#17
DF-BUG-16	candidate	`familyAndHousehold.xml` (qc-of-another answer)	Reads "qualifying child of another taxpayer?" only when the person passes this filer's age test and QR/QC tests; else treats as no. Wrong e.g. for a 21-year-old student sibling of a 20-year-old filer; QSS rule reads it without the gross income test	IRC 152(c)/(d)(1)(D)	Use the answer whenever the person could be anyone's qualifying child; no oracle scenario differs	#17
DF-BUG-17	candidate	`eitc.xml` (combat-pay variants)	Variant formulas have no zero case: earned income $0 plus combat pay, or elected combat pay pushing income past the completed phase-out, pays $1-2	IRC 32: credit is $0	Follow the statute; no oracle scenario affected	#26
DF-BUG-18	candidate	`dependentsBenefitSplit.xml`	Splits without a multiple support agreement	IRC 152(e)(5): the Form 8332 split does not apply under a multiple support agreement	Added input `multiple_support_agreement`; no oracle scenario affected	#18
DF-BUG-19	candidate	`familyAndHousehold.xml` / `filingStatus.xml` (considered unmarried)	Married filer is "considered unmarried" if the relationship is a child, without checking the child is actually claimable	IRC 7703(b)(1): only through a child the taxpayer can claim	Follow the statute; 1 oracle person differs; also changes the chosen status for 2 DF filing-status test graphs (HoH allowed by DF, knocked out here)	#19, #20

ID	Status	Where (DF XML)	What DF does	What the law says	Our behaviour	Owner
DF-BUG-20	candidate	`familyAndHousehold.xml` (HoH/QSS qualifying persons)	Requires the eligible dependent to be claimed before they qualify the filer for HoH or QSS	Law requires only that the filer could claim them; EITC and CDCC use the same "must be claimed" rule	Dropped the requirement; no oracle value changes	#19
DF-BUG-21	candidate	`familyAndHousehold.xml`	A married qualifying child is excluded also for no TIN or a custody release	IRC 2(b)(1)(A)(i): excluded only for filing jointly or failing the citizen test	Follow the statute; no oracle value changes	#19
DF-BUG-22	candidate	`familyAndHousehold.xml` (HoH other-relative path)	A non-custodial relative the filer cannot claim because of a custody split	Law requires one they can claim	Follow the statute; no oracle value changes	#19
DF-BUG-23	candidate	`cdcc.xml` (custodial-parent split)	A grandparent/ foster parent gets CDCC through the Form 8332 split: counts a foster child's $500 expenses (`hoh-cdcc-multiple-careproviders`: credit 702 vs 602)	IRC 21(e)(5), i2441 p1, Pub 503 p4: only the custodial parent	Follow the statute; 6 oracle facts diverge (known)	#28
DF-BUG-24	candidate	`cdcc.xml` (percentage)	AGI <= 0 gives a decimal amount of 0	Statute: 35%	Follow the statute; 3 facts in 3 scenarios, no credit changes	#28
DF-BUG-25	candidate	`cdcc.xml` (MFS unmarried exception)	Tests two possibly different persons	IRC 21(e)(4)	Statute; no oracle effect	#28
DF-BUG-26	candidate	`cdcc.xml` (QSS)	Qualifying surviving spouse's line 19 comes out as $0	Statute	Statute; no oracle effect	#28
DF-BUG-27	candidate	`cdcc.xml`	Forfeited benefits ignored unless W-2 box 10 is positive	Form 2441 Part III	Statute; no oracle effect	#28
DF-BUG-28	candidate	`cdcc.xml`	Requires the filer to have chosen to claim the child	Law requires only that the child be a qualifying person	Statute; no oracle effect	#28
DF-BUG-29	candidate	`eitc.xml` / `familyAndHousehold.xml` (EITC qualifying child)	Requires the child to be claimed by the filer	IRC 32(c)(3): requires only that the filer be entitled to claim the child	Statute; no oracle value changes	#48
DF-BUG-30	candidate	`eitc.xml` (EITC child test)	Second claimed-vs-entitled gate in the child test	IRC 32(c)(3)	Statute; no oracle value changes	#48

ID	Status	Where (DF XML)	What DF does	What the law says	Our behaviour	Owner
DF-BUG-31	candidate	`ptc.xml` (coverage months)	A policy starting December 1 never counts December as a coverage month; every other month allows a start on the 1st	Form 8962: a policy in effect on the 1st counts	Statute; no oracle value changes	#33
DF-BUG-32	candidate	`ptc.xml` (annual vs monthly)	Uses annual totals (line 11) whenever the policy starts in January and ends in December, even with fewer than 12 coverage months (Jan 20 start: $6,000 DF, $5,500 here)	Form 8962 instructions: annual only if covered all 12 months	Statute; no oracle value changes	#33
DF-BUG-33	candidate	`ptc.xml`	Uses annual totals if any 1095-A qualifies	Instructions: each	Statute; only matters with 2+ 1095-As, which DF blocks	#33

Add rows as modules find more; keep one row per distinct defect.

JSON API contract (issue #49)

Contract version: `api_version` 1, schema file `api/schema.json` (JSON Schema draft 2020-12, `schema_version` 1.2.0), tax year 2024. Consumers: the Lean server (#51 validation, #52 endpoint), the React frontend (#72 to #82). Plain HTTP; TLS is terminated by a proxy (`docs/lean-http-server.md`).

Files:

File	Role
`api/schema.json`	The contract. Request, success, unsupported, error, health and version shapes in `$defs`.
`api/knockout-reasons.json`	Every `UnsupportedReason.code` with category and Direct File fact. Generated from the Catala enum (`make api-reasons`).
`api/line-catalog.json`	Key of each `lines` entry to Form 1040 line, label, Catala output, DF fact.
`api/examples/*.json`	Request and response pairs: one per filing status, refund, balance due, W-2 plus interest, a family with two children, earned income credit, knocked out. Every response is the output of the Catala `ApiCalculator` (`make api-examples`).
`docs/api-mapping.md`	The reference mapping from this contract to the Catala `Calculator` inputs (issue #38): field tables, decoding and encoding rules, the defaults.
`api/fixtures/manifest.json`	Mock-server table (method, path, request, status, response) for the frontend (#74): supported, unsupported and every error status.
`make api-check`	Validates all of the above against the schema (below).

Endpoints

Method and path	Success	Failure
`POST /calculate` (`Content-Type: application/json`, body at most 256 KiB)	200 `CalculateSuccess`	422 `CalculateUnsupported`; 400, 413, 415, 500 `ErrorResponse`
`GET /health`	200 `{"status":"ok"}`	
`GET /version`	200 `VersionResponse` (`api_version`, `schema_version`, `tax_year`, `calculator_version` = git commit)	
anything else		404 `not_found`, or 405 `method_not_allowed` for a known path with the wrong method

Every body is `application/json; charset=utf-8`. The server holds no state: the same request gives the same answer.

Money: integer cents

Every money value, in requests and responses, is a **JSON integer number of cents** (`$50,000.00` is `5000000`). Reasons:

- Lossless and identical to the verified core: Catala `money` and Lean `Money` are integer cents (`catala/CONVENTIONS.md` section 4), so the server converts with no arithmetic and no rounding.

- No float ever appears on the wire. A decimal string (`"50000.00"`) would need a parser with rules for leading zeros, signs, exponents and one or two decimals; an integer needs none.
- JavaScript reads it exactly: the schema caps `|value| <= 10^13` cents ($100 billion), far below 2^53, so `JSON.parse` and `number` are safe and no BigInt is needed. Sums over the 20-item list caps stay below 2^53.

Rules for the validator (#51): a money field is accepted only as an integer literal. `1.5`, `1e2` and `100.0` are rejected with `not_integer` (JSON Schema's `integer` also admits `100.0`; the server must be stricter), as are strings and values outside the range. Fields that cannot be negative on the form use `NonNegMoney`; only `social_security.net_benefits` (SSA-1099 box 5 may be negative) uses signed `Money`.

Rounding is entirely the calculator's, per CONVENTIONS section 4: the server never rounds an input. Inputs may carry cents; Direct File rounds each box to whole dollars where its XML does (`Round`, ties away from zero), and so does the verified code. As a consequence every output line is a whole-dollar amount, a multiple of 100 cents. The frontend must display the returned integers and must not recompute or re-round them. Tax Table lookups use the row midpoint (`R.stepwise_multiply`), so for example taxable income $35,400 gives $4,019.

Request: `POST /calculate`

The request holds the writable facts the Catala top-level scope reads (`ApiCalculator`, which calls `Calculator`; `docs/api-mapping.md` has the field-by-field mapping), with box-oriented names. Required: `api_version` (1), `tax_year` (2024), `filing_status`. Everything else is optional; an absent list is empty, an absent money box is $0, an absent flag is `false` (CONVENTIONS section 3: inputs are complete, so absent means the value Direct File would use).

```
{
  "api_version": 1,
  "tax_year": 2024,
  "filing_status": "Single",
  "w2s": [{ "filer": "PrimaryFiler", "wages": 5000000, "federal_withholding": 500000 }]
}
```

Property	Type	Catala input
`filing_status`	`Single`, `MarriedFilingJointly`, `MarriedFilingSeparately`, `HeadOfHousehold`, `QualifyingSurvivingSpouse`	`filing_status` (`Types.FilingStatus`)
`filers.primary` / `filers.secondary`	`{is_blind, age65_or_older, can_be_claimed_as_dependent}` and, since 1.2.0, `date_of_birth`, `will_be_claimed`, citizenship answers, `is_student`, `tin`, `disability`, `hsa`, `cdcc`	`FilerAnswers` and `FilerExtra` of `Calculator`
`filing_details`	`treat_as_mfs`, `is_widowed_in_tax_year`, `mfj_deceased_spouse_reached_age65`, `mfs_spouse_65_or_older`, `mfs_spouse_provides_no_benefits`, `treat_filers_as_dependents`, `has_foreign_accounts_or_trusts`, `combat_pay_election`, `combat_pay_election_made`	the 1.0 flags are folded into the filers' facts (`docs/api-mapping.md`)

Property	Type	Catala input
`filing_answers` (1.2.0)	marital status, spouse's death, living apart, residence state, MFS answers	`Calculator.FilersInput`
`dependents[]` (1.2.0)	`Dependent`: the Direct File per-person facts of a household member	`Calculator.HouseholdMember`
`credits`, `cdcc`, `premium_tax_credit`, `screening` (1.2.0)	the credit, child care, premium tax credit (with Forms 1095-A) and screening answers, named as in Catala	`credits`, `cdcc_answers`, `ptc_answers`, `screening`
`w2s[]`	`FormW2`: boxes 1 to 11, `box12` object keyed by code letter, box 13 flags	`w2s` (`Form_w2s.FormW2`)
`interest_reports[]`	`Form1099Int`	`interest_reports` (`Interest.InterestReport`)
`form_1099g[]`	`Form1099G`	`Unemployment.Form1099G`
`form_1099misc[]`	`Form1099Misc`	`Form1099_misc.Form1099Misc`
`form_1099r[]`	`Form1099R`	`Form1099_rs.Form1099R`
`social_security`	`{net_benefits: [Money], federal_tax_withheld: [NonNegMoney] (1.2.0), lived_apart_all_year}`	`Social_security` inputs
`adjustments`	`educator_expenses`, `student_loan_interest`, `hsa` (1.2.0: Form 8889 answers and distributions)	`Educator_adjustment`, `Student_loan_adjustment`, `Hsa` inputs
`estimated_payments`	`{paid_estimated_taxes_or_from_last_year: amount, paid_under_different_name, paid_with_former_spouse}`	`Estimated_payments` inputs

The mapping from each API key to the exact Catala field (for example W-2 `wages` is `writable_wages`, 1099-INT `federal_tax_withheld` is `writable_tax_withheld`) is in the `description` of the matching `$defs` entry. Naming follows the form box, not the Direct File fact path, so the UI and a paper form agree; the conversion function in #51 is the single place that knows both.

Validation policy (for #51)

- **Unknown properties are rejected** (`additionalProperties: false` everywhere) with HTTP 400 `unknown_field`, never ignored: a typo must not silently drop income.
- `api_version` other than 1 gives `unsupported_api_version`; `tax_year` other than 2024 gives `unsupported_tax_year`.
- Beyond the schema: `SecondaryFiler` is allowed only for `MarriedFilingJointly` (`filer_not_allowed`); at most one `SecondaryFiler` and one `PrimaryFiler` filer object, and no `filers.secondary` unless joint; dates must exist (`invalid_date`); lists hold at most 20 items (`too_many_items`; `dependents` too); `social_security.federal_tax_withheld` is no longer than `net_benefits` (`range`); box 12 codes are the letters in the schema.
- Implemented in Lean (#51, `lean/UsTax/Api`, spec `Request.Valid`); `make api-conformance` cross-checks it against this schema. The Lean reader is stricter than ajv in three documented ways: integer-valued float literals (`100.0`, `1e2`) are `not_integer`, duplicate object keys are `invalid_json`, and `SecondaryFiler`/impossible dates are refused (the schema cannot express them). Shape problems (phase 1) are reported before range/size/role problems (phase 2); a body with both shows the shape problems first.
- Duplicate persons (`duplicate_person`) apply once the household section lands.

- The body cap is 256 KiB: the server catches the `readAll` failure and answers 413.

Coverage of the writable facts

Since schema 1.2.0 (issue #38) the calculator behind the API is the whole-return Catala scope `Calculator`: every section the contract accepts is read, none is "accepted but not used". The only requests whose sections are not all read are the ones that are knocked out (422), which stop at the reasons.

Writable facts	Status
Filing status, W-2s, 1099-INT, foreign accounts, combat pay	In API, computed
Per-filer blind, 65 or older, can be claimed, date of birth, citizenship; MFS/widow/dependent answers	In API, computed
1099-G, 1099-MISC (Alaska PFD), 1099-R, SSA-1099, educator and student loan adjustments, estimated payments	In API, computed
HSA (Form 8889 answers, distributions)	In API since 1.2.0 (`adjustments.hsa`, `filers.*.hsa`), computed
Household: qualifying children, qualifying relatives, dependents (`dependents[]`), earned income credit, child tax credit and credit for other dependents, child and dependent care credit (`cdcc`), Saver's credit, elderly or disabled credit (`credits`), premium tax credit with Forms 1095-A (`premium_tax_credit`)	In API since 1.2.0, computed
Screening questions (digital assets, foreign trusts, IRA contributions, state questions)	In API since 1.2.0 (`screening`); they only feed knockouts
Itemized deductions, Schedule C/D/E, capital gains, dividends	Not supported by Direct File; excluded permanently. Sending them is `unknown_field`; situations that need them are knockouts (`IncomeSourcesNotSupported`)
Name, address, SSN, bank account, dates of birth (except filers', dependents' and the 1099-R recipient's)	Excluded: no tax effect; the calculator takes no personal data beyond the dates of birth the age tests need
Process answers (`flowHasSeenDeductions`, too-late-to-file)	Part of `screening` where a knockout reads them; the rest is decided server side or by the frontend flow

Defaults for what a request leaves out are in `api/schema.json` (`default`) and in `docs/api-mapping.md`. A client that collects an answer should send it: a default is the ordinary-filer answer, not a guess about the person.

Response 200: `CalculateSuccess`

```
{
  "api_version": 1, "tax_year": 2024, "supported": true, "reasons": [],
  "outcome": "balance_due",
  "lines": { "wages": 5000000, "agi": 5000000, "standard_or_itemized_deductions": 1460000,
             "taxable_income": 3540000, "total_tax": 401900, "total_payments": 0,
             "overpayment": 0, "balance_due": 401900, "...": "see api/examples/single-
w2-50k.response.json" }
}
```

- `outcome` is `refund` (line 34 > 0), `balance_due` (line 37 > 0) or `even`; at most one of `overpayment` and `balance_due` is non-zero.
- `lines` has one integer-cents entry per scalar output of the Catala `Form1040` scope that the Lean mirror implements: `wages` (1a), `employer_income_subtotal` (1z), `interest_income` (2b), `total_income` (9), `adjustments_to_income` (10), `agi` (11), `standard_or_itemized_deductions` (12), `total_deductions` (14),

`taxable_income` (15), `tentative_tax_from_taxable_income` (16), `total_tentative_tax` (18), `non_refundable_credits` (21), `tax_less_non_refundable_credits` (22), `total_tax` (24), `total_withholding` (25d), `total_other_payments_refundable_credits` (32), `total_payments` (33), `overpayment` (34), `balance_due` (37). `api/line-catalog.json` gives label, form line, Catala output and DF fact for each, for the line-by-line view (#77) and rule links.

- Extensibility: new lines (Schedule 1/2/3 and 8812 values, credits, `qualified_business_income_deduction`) are added as new keys. Clients ignore keys they do not know; `line-catalog.json` says how to label them. A future optional `details` object may carry the "how we got this number" breakdown.

Warnings (schema 1.1.0, #52; 1.2.0)

`CalculateSuccess` may carry an optional `warnings` array: `{code, path, message}` per request part that validated but did not affect the result. `path` is an RFC 6901 pointer such as `/filing_details/treat_as_mfs`. It is omitted when empty, so a typical request gets exactly the body shown above. Codes: `section_not_used` (1.1.0: a section whose module was not wired yet; none since 1.2.0, every section is read) and `field_superseded` (1.2.0: a 1.0 `filing_details` flag that the calculator now derives from the filers' facts, see `docs/api-mapping.md`). The frontend should show the warning next to the result.

Response 422: `CalculateUnsupported`

At least one knockout fired, so no return is computed (CONVENTIONS section 6; Direct File's `/flowIsKnockedOut`). Status is 422: the request is well formed but describes a situation outside the supported subset. There is no `lines` property.

```
{ "api_version": 1, "tax_year": 2024, "supported": false,
  "reasons": [{ "code": "FlowKnockoutHasBonds", "category": "interest", "message": "Flow
Knockout Has Bonds" }] }
```

- `reasons` is non-empty and lists every knockout that fired, in the order of the Catala `KnockoutReason` enumeration (`fired_knockouts` of `KnockoutsIncome`, issue #36).
- `code` is the `KnockoutReason` constructor name. The valid codes, with category and DF fact, are in `api/knockout-reasons.json`; `make api-reasons` regenerates it from `catala/knockouts_income.catala_en` and `docs/knockout-inventory.md`, and `make api-check` fails if it is stale.
- `category` groups codes for the frontend copy screens (#80): `credits`, `estimated_payments`, `filers`, `health_coverage`, `household`, `hsa`, `income`, `interest`, `process`, `retirement`, `social_security`, `state`, `unemployment`, `w2`. The module-to-category map is in `api/scripts/gen-reasons.mjs`.
- `message` is English fallback text (the constructor name split into words). The frontend owns the user-facing copy and keys it by `code`; for an unknown `code` it shows its category text.

How the calculator plugs in (issue #38)

The Catala scope `ApiCalculator` (`catala/api_calculator.catala_en`) is the reference for the response: its outputs `supported`, `reasons`, `outcome` and `lines` are the response fields (`docs/api-mapping.md` has the decode and encode rules). `supported = not unsupported`; when false, `reasons` is the list of `CalculatorKnockout` constructors with their categories, and the response is 422 without `lines`. `api/knockout-reasons.json` lists every code (the household, credit and premium tax credit reasons are `Extra...` constructors of `KnockoutReason`); `make api-reasons` regenerates it together with the reason to category function of the Catala side (`catala/api_reasons.catala_en`). Reasons are additive (a minor version).

Errors: `ErrorResponse`

```
{ "api_version": 1,
  "error": { "code": "invalid_request", "message": "The request body is invalid.",
             "details": [{ "path": "/w2s/0/wages", "code": "range", "message": "must be >=
0" }] } }
```

HTTP	`error.code`	When
400	`invalid_json`	body is not JSON
400	`invalid_request`	shape, type, enum, range, integer-ness, cross-field rules; `details` lists every problem (RFC 6901 `path`)
400	`unknown_field`	an unknown property (also listed in `details`)
400	`unsupported_api_version`, `unsupported_tax_year`	version or year not served
404 / 405	`not_found`, `method_not_allowed`	wrong path or method
413	`payload_too_large`	body over 262144 bytes
415	`unsupported_media_type`	`Content-Type` is not `application/json`
500	`internal_error`	server fault; no tax logic is exposed

Unsupported (422) is deliberately a different shape from errors: it is an expected, user-facing outcome, not a fault. A client switches on `status`, then on `supported` or `error.code`.

Versioning

- `api_version` is the major version, sent in every request and response; the server rejects other values.
- Additive changes (new optional request property, new response `lines` key, new reason code, new error code) bump the minor part of `schema_version` and keep `api_version`. Clients must ignore unknown response keys and show a generic message for unknown reason codes.
- Changing or removing a property, making an optional property required, or changing a unit bumps `api_version` (and the major of `schema_version`); the server then serves both for a transition.
- Request validation is strict, response consumption is lenient (Postel in the direction that protects tax data).

Checking the contract

```
make api-check     # cd api && npm ci --ignore-scripts && node scripts/check.mjs
make api-reasons   # regenerate api/knockout-reasons.json and catala/api_reasons.catala_en
after the Catala enums change
make api-examples  # recompute every example/fixture response with the Catala ApiCalculator
(needs clerk)
```

`api-check` (Node 18 or later; `ajv` 8.17.1 pinned in `api/package-lock.json`, installed under `api/node_modules`, not committed) validates:

1. every response in `api/fixtures/manifest.json` against the schema type for its status (200, 422, other, health, version), and every request that should be valid against `CalculateRequest`; requests of 400 fixtures must violate the schema, except those marked `semantic_error`;
2. arithmetic invariants of each success response (AGI, taxable income floor, refund versus balance due, `outcome`, no negative line);

3. that every reason `code` and `category` exists in `api/knockout-reasons.json`, that file is fresh against the Catala enum, and the schema's `Lines.required` equals `api/line-catalog.json`;
4. one example per filing status, and an example file for every status class (200, 400, 413, 422).

The numbers in the examples are the output of the Catala scope `ApiCalculator` (`make api-examples` recomputes them) and agree with the hand-worked returns of `catala/tests/test_calculator.catala_en` (Tax Table midpoints): Single $50,000 gives tax $4,019, joint $70,000 gives $4,435, the family of four $1,635 after the $4,000 child tax credit, the head of household with one child a $3,604 refund (earned income credit $1,448, additional child tax credit $156). The unsupported examples show the response shape; their exact reason lists are confirmed by `make test-server` (`scripts/server_integration.mjs`). That test found that box 3 (`government_bond_interest`) does not fire `FlowKnockoutHasBonds` (only boxes 9 to 13 do, `catala/interest.catala_en`), so `knocked-out-bond-interest.request.json` now uses box 9 (`private_activity_bond_interest`).

Examples (curl)

```
curl -s -X POST http://localhost:8080/calculate -H 'Content-Type: application/json' \
  --data @api/examples/single-w2-50k.request.json     # 200, balance_due 401900
curl -s -X POST http://localhost:8080/calculate -H 'Content-Type: application/json' \
  --data @api/examples/knocked-out-bond-interest.request.json   # 422, FlowKnockoutHasBonds
```

Example	Result
`single-w2-50k`	AGI 5000000, standard deduction 1460000, taxable income 3540000, tax 401900, balance due 401900
`refund-single-w2-50k`	$5,000 withheld: overpayment 98100
`balance-due-single-w2-50k`	$3,000 withheld: balance due 101900
`mfj-two-w2s`	Joint, two W-2s, tax 443500
`mfs-w2-40k`, `hoh-w2-60k`, `qss-w2-80k`	One per remaining filing status; head of household and surviving spouse each have one child (`dependents[]`), since those statuses need a qualifying person
`family-mfj-two-kids`	Joint, $80,000 of wages, two children: child tax credit $4,000, tax 163500, refund 136500
`eitc-hoh-one-child`	Head of household, $40,000 of wages, one child: tax 0, earned income credit $1,448 plus additional child tax credit $156 (line 32 160400), refund 360400
`refund-below-standard-deduction`	$10,000 of wages, no children: tax 0, earned income credit $632 (line 32 63200), refund 113200
`knocked-out-hoh-no-qualifying-person`	422, `ExtraFilingStatusNotEligible` (head of household without a qualifying person)
`single-w2-and-interest`	W-2 plus 1099-INT, tax 413900 (oracle scenario `single-50k`)
`knocked-out-bond-interest`	422, `FlowKnockoutHasBonds`

Aggregate income limit

The endpoint rejects a return with taxable income above $1 trillion with HTTP 400 `invalid_request` and a `range` detail at the document root. Individual field caps do not bound the sum of multiple forms. This guard keeps successful responses within the Tax Computation Worksheet proof domain; it does not change `Request.Valid` or its validation theorem.